

PulseGuard — Interview Defense Document

Version: V3 — Gold Pass 4/4 + Deep Drill + Role Expansion | **Date:** 2026-06-18

Sourced exclusively from repo files and evidence artifacts

This document exists to prepare the author for any technical interview question about PulseGuard. Every metric, claim, and limitation is traceable to an artifact in ``outputs/evidence/``.

Label Legend

Label	Meaning
<code>`[IMPLEMENTED]`</code>	Code exists, runs, and produces a verified artifact or measured output
<code>`[PARTIALLY VERIFIED]`</code>	Code runs but validation is limited by public data, proxy fields, or sample constraints
<code>`[SIMULATED]`</code>	Deliberately scenario-based or synthetic — e.g. cost matrix, policy demo
<code>`[FUTURE]`</code>	Not built; explicitly absent or deferred

Section 1 — Project Identity and Honest Framing

What PulseGuard Is

PulseGuard is a **credit-risk model governance portfolio project**. It demonstrates the complete ML governance lifecycle on the Home Credit Default Risk public dataset: raw multi-table data ingestion, feature engineering across 7 relational tables, leakage-audited train/val/test splits, a 12-model baseline tournament, validation-only Optuna hyperparameter tuning, post-hoc Platt calibration, a 9-component composite champion selection, score-band policy design, SHAP reason-code explainability with bootstrap stability, a fairness proxy audit skeleton, a drift/PSI monitoring baseline, and a local BM25+LLM governance assistant for adverse action drafting.

What PulseGuard Is Not

- A production lending system
- A regulatory-compliant credit scoring deployment
- A legally certified adverse action notice generator
- A model approved for underwriting in any jurisdiction
- A system trained on real bank customer data
- A fairness-certified model under ECOA or any applicable regulation
- A system with real out-of-time validation (no timestamps in Home Credit)

Strongest Safe One-Liner

"End-to-end credit-risk governance stack: tuned LightGBM on 307k Home Credit applicants, calibrated PD scores (ECE=0.0034), score-band policy, SHAP reason codes stable across 30 bootstrap resamples, fairness proxy audit, and a local RAG/LLM governance assistant — all under ASSISTIVE_ONLY, HUMAN_REVIEW_REQUIRED constraints."

Why Credit-Risk Decisions Are High-Stakes

Credit decisions are legally regulated in most jurisdictions. ECOA (US) and equivalent laws require adverse action notices with specific reasons. SR 26-2 (April 2026, Fed/FDIC/OCC) mandates governed model lifecycle management — not just model development but champion selection, ongoing monitoring, human review protocols, and explicit limitation documentation. A poorly governed credit model can deny credit to qualified applicants or extend credit to defaulters at scale.

Why This Is Governance, Not Just a Model

PulseGuard is explicitly designed to demonstrate model governance: the model is only one layer. The project also produces a champion selection rationale (not AUC alone), calibrated probabilities with ECE documentation, score-band policy with semantic justification, SHAP reason codes with stability evidence, a fairness skeleton with documented limitations, a drift monitoring baseline, a RAG/LLM assistant with hard ASSISTIVE_ONLY constraints, and a full evidence ledger mapping every claim to a file.

What to Say in Interview

"PulseGuard is my credit-risk governance portfolio project. I built it to demonstrate that I understand the full ML lifecycle — not just model training, but leakage control, calibration, score-band policy, explainability, fairness awareness, and governance documentation. The champion is LightGBM with monotone constraints, calibrated to ECE=0.0034, selected by a 9-component composite score. All limitations are documented in the model card."

What Not to Say

- "I built a production credit scoring system" — it is a portfolio project
 - "The model is fair" — proxy audit only; no protected-class labels
 - "AUC=0.77 exceeds industry benchmarks" — no benchmark specified; claim is undefended
 - "The LLM makes credit decisions" — ASSISTIVE_ONLY; human decides
-

Section 2 — Dataset and Scope

Home Credit Default Risk (PRIMARY) `[IMPLEMENTED]`

Field	Value
Source	Kaggle — Home Credit Default Risk public competition
Applicants	307,511
Default rate	8.07% (scale_pos_weight = 11.39)
Total rows	57.4M across 7 side-tables
Geography	Single Eastern European portfolio
Timestamps	None
Split	Stratified random 60/20/20, seed=42, DR preserved across all splits

Tables used:

- `application\_train.csv` — core applicant fields (base table)
- `bureau.csv` + `bureau\_balance.csv` — credit bureau history and monthly balances
- `previous\_application.csv` — prior Home Credit applications
- `installments\_payments.csv` — instalment payment history
- `credit\_card\_balance.csv` — revolving credit card monthly snapshots
- `POS\_CASH\_balance.csv` — POS loan / cash advance history

Taiwan Default (LEGACY_APPENDIX_ONLY)

Used at G6/G7 as a bridge dataset. Not used in Pass 2–4 modeling. Retained for historical context only. Not to be reintroduced as primary.

LendingClub (DROPPED)

Dropped from current scope. Reject inference is a separate workstream.

Why Home Credit Is the Better Primary Spine

- 10× more applicants than Taiwan Default (307k vs 30k)
- 7 relational side-tables enable deep feature engineering (instalment ratios, bureau aggregates, POS DPD)
- Real external credit bureau signals (EXT_SOURCE_1/2/3) as primary risk drivers
- 8.07% base rate matches real consumer-loan portfolios (vs 22% for Taiwan credit cards)
- SHA256 provenance verified — reproducible public download

Public-Data Limitations

- No temporal ordering — out-of-time split not possible
- Approved applicants only — reject inference unaddressed (MNAR selection bias)
- Single geography — no cross-market generalisation claim
- No real protected-class labels — fairness audit limited to proxies
- No income field — FOIR is a proxy (credit amount / goods price)

Section 3 — Architecture Overview

Stage	Description	Status	Artifact
Data audit	8-table row counts, DR, scale_pos_weight, temporal feasibility	`[IMPLEMENTED]`	`g9a_home_credit_data_audit.json`
Feature factory	140 features from 7 tables: ratios, aggregates, composites	`[IMPLEMENTED]`	`g9a_feature_factory_report.json`
Leakage audit	10/10 checks: TARGET exclusion, val-only fit, no test contamination	`[IMPLEMENTED]`	`gold_pass1_leakage_audit.json`
Train/val/test split	Stratified 60/20/20, seed=42, DR=0.0807 preserved	`[IMPLEMENTED]`	`g9a_splits.pkl` verified
Baseline tournament	12 models, 2 hard-failures, provisional champion CatBoost	`[IMPLEMENTED]`	`g9a_model_tournament_report.json`
Hyperparameter tuning	Optuna TPE, 5 models, validation-only, 4–8 trials	`[IMPLEMENTED]`	`gold_pass2_tuning_trace.json`
Calibration	Platt sigmoid, val-only fit, Isotonic excluded	`[IMPLEMENTED]`	`gold_pass2_calibration_report.json`
Champion selection	9-component composite; LightGBM_mono wins	`[IMPLEMENTED]`	`gold_pass2_champion_selection_report.json`
Final test evaluation	Single evaluation on 61,503-row test set	`[IMPLEMENTED]`	`gold_pass2_final_untouched_test_report.json`
Threshold/score bands	GREEN/AMBER/RED; PD-semantic; cost-sensitive	`[IMPLEMENTED]`	`gold_pass3_threshold_score_band_report.json`
SHAP reason codes	pred_contrib; 20 global + 4 local cases; ECOA drafts	`[IMPLEMENTED]`	`gold_pass3_shap_reason_code_report.json`
Reason-code stability	30 bootstraps; top-5 features 30/30 stable	`[IMPLEMENTED]`	`gold_pass3_reason_code_stability.json`

Stage	Description	Status	Artifact
Fairness/proxy audit	Age, income, employment, region proxy groups	`[PARTIALLY VERIFIED]`	`gold_pass3_fairness_proxy_audit.json`
Drift/PSI baseline	val-vs-test PSI=0.0002; feature PSI STABLE	`[PARTIALLY VERIFIED]`	`gold_pass3_drift_vintage_stress.json`
RAG/LLM governance	6-case BM25+LLM demo, abstain, ASSISTIVE_ONLY	`[SIMULATED]`	`gold_pass3_rag_llm_demo_report.json`
Human review	Override protocol, officer sign-off requirement	`[SIMULATED]`	`gold_pass3_rag_llm_demo_report.json` + governance report

Section 4 — Feature Engineering

`[IMPLEMENTED]` — 140 features from 7 source tables after constant/near-zero-variance removal.

Core Ratio Features (application_train)

- `CREDIT\_TO\_INCOME` — total credit / reported income (FOIR proxy)
- `CREDIT\_TO\_GOODS` — credit amount / goods price (LTV proxy)
- `CREDIT\_TO\_ANNUITY` — credit amount / annual instalment (repayment capacity)
- `ANNUITY\_TO\_INCOME` — proposed annual repayment / income (debt service ratio)
- `AGE\_YEARS` — applicant age (–1 monotone constraint: older → lower risk)
- `EMPLOYED\_YEARS` — tenure in years (–1 monotone constraint; sentinel fix for unemployed)
- `FLAG\_EMPLOYED\_ANOMALY` — binary flag when DAYS_EMPLOYED=365243 (HC encoding for not employed)

Bureau Aggregates (bureau + bureau_balance)

- `BUREAU\_ACTIVE\_COUNT` — number of active bureau credit lines
- `BUREAU\_OVERDUE\_RATIO` — overdue accounts / total accounts (+1 monotone constraint)
- `BUREAU\_AMT\_OVERDUE` — total outstanding overdue balance (+1)
- `BB\_DPD\_RATIO\_MEAN` — mean DPD ratio across bureau balance records (+1)

Previous Application Aggregates

- `PREV\_REFUSAL\_RATE` — fraction of prior HC applications refused (+1)
- `PREV\_AMT\_CREDIT\_MEAN` — average prior credit amount requested

Instalment Payment Features

- `INST\_LATE\_RATIO` — late payments / total payments (+1 constraint; rank-4 SHAP feature)

Credit Card Features

- `CC\_DPD\_RATIO` — credit card days-past-due ratio (+1)
- `CC\_ATM\_RATIO` — ATM withdraw / total drawings

POS Cash Features

- `POS\_IS\_DPD\_RATIO` — POS instalment days-past-due ratio (+1)

Composite Signal

- `BEHAVIORAL\_RISK\_SCORE` = $0.4 \times \text{INST_LATE_RATIO} + 0.3 \times \text{CC_DPD_RATIO} + 0.2 \times \text{POS_IS_DPD_RATIO} + 0.1 \times \text{BUREAU_OVERDUE_RATIO}$

Leakage Controls

- `TARGET` excluded from all feature sets (confirmed by 10/10 leakage audit)
- Post-outcome bureau fields checked; DAYS_DECISION in side-tables controlled
- Group leakage: 0 SK_ID_CURR overlap between train and test confirmed

Section 5 — Model Tournament

Baseline Tournament (G9A) `[IMPLEMENTED]`

12 models with near-default hyperparameters:

Model	Val AUC	Status
CatBoost + Platt	0.7716	Provisional baseline champion
XGBoost + Platt	0.7703	Contender
LightGBM_base + Platt	0.7203	Baseline (bug: early stopping)
LightGBM_monotonic + Platt	0.7203	Baseline (bug: early stopping)
RF, LR, SVM	0.65–0.72	REJECTED
TabNet	HARD_FAIL	CPU ~6min/epoch; estimated 400–800h on 184k rows
sklearn GBM	HARD_FAIL	Wall-time exceeded

Tuned Tournament (Gold Pass 2) `[IMPLEMENTED]`

Model	Val AUC	Composite	Role
LightGBM_monotonic	0.7734	0.7312	CHAMPION + GOVERNANCE
XGBoost_monotonic	0.7699	0.7294	Contender
LightGBM_base	0.7724	0.6811	Contender (no governance premium)
CatBoost	0.7708	0.6802	Contender (no monotone constraints)
XGBoost	0.7704	0.6801	Contender

Why Baseline Was Not Enough

The G9A baseline was intentionally run with near-default parameters as a pre-tuning audit. The purpose was to establish competitive ordering and confirm leakage-free pipeline — not to find the best model. Gold Pass 1 confirmed `safe\_to\_tune=true` before any HPO. The baseline also revealed the LightGBM early-stopping bug, which would have produced incorrect results if tuning had proceeded before fixing.

Why Tuning Was Necessary

LightGBM_monotonic at 0.7203 (baseline) vs 0.7734 (tuned) — a +0.053 gap. The baseline underrepresented LightGBM because the early-stopping bug (scale_pos_weight + eval_metric='auc' fires at iteration=1) caused the model to stop at 1 tree. Fixing this required treating n_estimators as an Optuna hyperparameter rather than using native early stopping.

Section 6 — Hyperparameter Tuning `[IMPLEMENTED]`

Field	Value
Algorithm	Optuna Tree-structured Parzen Estimator (TPE)
Seed	42
Budget	35-second wall-clock per model (CPU-only sandbox, 44s bash limit)
Trial counts	LightGBM_base: 6 · LightGBM_mono: 5 · CatBoost: 8 (2 runs) · XGBoost: 4 · XGBoost_mono: 4
Champion selection	Validation set only — test set never touched during HPO
Calibration	Platt fit on val only — after HPO, before test evaluation
Test evaluation	Exactly once, after champion locked

Key interview note: Trial counts (4–8) are lower than the 100-trial plan in the tuning spec. CPU sandbox with 44s bash limit is the constraint. The planning document specifies 100 trials; actual counts are documented honestly in ``gold_pass2_tuning_trace.json``. Results represent a reasonable local optimum.

Section 7 — Champion Model `[IMPLEMENTED]`

LightGBM with Monotone Constraints + Platt Calibration

Field	Value
Model family	LightGBM Gradient Boosted Trees
Variant	15 monotone constraints (12 +1 risk-increasing, 3 -1 risk-decreasing)
Calibration	Platt sigmoid (LogisticRegression C=1e6, fit on val only)
Composite score	0.7312 (9-component)

Val metrics: AUC=0.7734, PR-AUC=0.2661, KS=0.4121, ECE_platt=0.0051

Test metrics: AUC=0.7769, PR-AUC=0.2628, KS=0.4141, Brier=0.0668, ECE_platt=0.0034

Why selected by composite, not AUC alone:

CatBoost val_AUC=0.7708 is only 0.0026 below LightGBM_mono val_AUC=0.7734. But the composite includes 10% explainability weight (SHAP + monotone constraints = 1.0 for LightGBM_mono; 0.5 for models without monotone) and 5% adverse-reason-ready weight. These governance-focused weights make LightGBM_mono both performance champion and governance champion — eliminating the need for any trade-off.

Why monotonicity matters:

Monotone constraints guarantee directional interpretability auditable without per-applicant SHAP. A credit officer can verify: "for any two applicants identical in all features except BUREAU_OVERDUE_RATIO, the one with higher ratio will always have a higher predicted default probability." This is a SR 26-2-aligned interpretability property.

Section 8 — Calibration `[IMPLEMENTED]`

Isotonic regression calibration on validation-set probabilities:

Step	Detail
Calibrator	IsotonicRegression(out_of_bounds='clip') — piecewise-monotone interpolation
Fit data	Validation set only
Platt	Evaluated but not selected — sigmoid slope/intercept optimised for ECE but produced higher test ECE than isotonic

Step	Detail
Test ECE	0.0034 — near-production-quality calibration
Serving	Extracted as two numpy arrays (`iso_x.npy`, `iso_y.npy`) — `np.interp(raw_prob, iso_x, iso_y)` exactly replicates `IsotonicRegression.predict()` with zero sklearn dependency

Discovery note: The GP2 training report originally stated "Platt selected". Forensic inspection of `champion\_calibrated.pkl` (`cal['selected']`) confirmed isotonic was actually selected. The isotonic calibrator was extracted to numpy arrays for the Cloud Run deployment — eliminating sklearn version lock at serve time entirely.

Why calibrated PD matters more than raw score:

A raw GBM score is not a probability — it's an uncalibrated log-odds output. After isotonic calibration, PD=0.20 means the model genuinely estimates a 20% default probability. This enables: (1) semantically defensible score-band thresholds, (2) cost-sensitive threshold formula ($\theta^* = C_{\text{reject}} / (C_{\text{bad}} + C_{\text{reject}})$), (3) probabilistic adverse-action language, (4) ECE as a measurable governance metric.

Section 9 — Thresholds and Score Bands `[IMPLEMENTED]`

Band	Threshold	Test %	Test DR	Interpretation
GREEN	PD < 0.20	89.72%	5.77%	Auto-approve eligible (subject to hard-stop check)
AMBER	$0.20 \leq \text{PD} < 0.40$	9.80%	26.96%	Manual credit officer review required
RED	PD ≥ 0.40	0.47%	53.77%	Enhanced underwriting or decline

PD=0.20 semantic justification: At this boundary the model estimates 20% default probability. The GREEN band observed DR (5.77%) is materially below 20%, confirming conservative assignment. The threshold is semantically defensible to an auditor: "applicants below this line have a predicted default probability under 20%."

Cost-sensitive analysis `[SIMULATED]`: Under scenario economics ($C_{\text{bad}}=10$, $C_{\text{reject}}=1$, $C_{\text{review}}=0.3$), the cost-optimal single threshold is 0.47 with expected loss 0.0807/applicant. These are scenario assumptions, not real bank economics. Operational threshold requires real LGD, NIM, and regulatory capital parameters.

Section 10 — SHAP Reason Codes `[IMPLEMENTED]`

Method: LightGBM built-in `pred\_contrib=True` (native SHAP implementation; avoids SHAP library pandas-index dependency).

Global feature importance (mean |SHAP|, 1000-sample draw):

Rank	Feature	Mean	SHAP		Monotone
1	EXT_SOURCE_M EAN	0.510	-1 (risk-decreasing)		
2	CREDIT_TO_ANN UITY	0.141	N/A		
3	CREDIT_TO_GO ODS	0.139	N/A		
4	INST_LATE_RATI O	0.129	+1 (risk-increasing)		
5	EXT_SOURCE_1	0.120	N/A		

Rank	Feature	Mean	SHAP		Monotone
6	OWN_CAR_AGE	0.092	N/A		
7	EMPLOYED_YEARS	0.091	-1		
8	BUREAU_ACTIVE_COUNT	0.087	N/A		
9	AMT_GOODS_PRICE	0.082	N/A		
10	EXT_SOURCE_3	0.078	N/A		

Local cases computed:

Case	PD	Band	Primary Driver	SHAP
GREEN_APPROVE	0.0335	GREEN	EXT_SOURCE_MEAN	-0.829
AMBER_REVIEW	0.2056	AMBER	EXT_SOURCE_MEAN	+0.939
RED_HIGH_RISK	0.4164	RED	EXT_SOURCE_MEAN	+1.300
AMBER_CONFLICT	0.3475	AMBER	EXT_SOURCE_MEAN	+1.232

Adverse-action-style reason codes (ECOA-draft, ASSISTIVE_ONLY):

- EXT_SOURCE_MEAN → "External credit bureau composite score below threshold"
- INST_LATE_RATIO → "High proportion of late instalment payments"
- CREDIT_TO_ANNUITY → "Credit amount high relative to annual repayment capacity"
- CREDIT_TO_GOODS → "Loan-to-goods-value ratio elevated"

These are draft aids, not legally compliant adverse action notices.

Section 11 — Reason-Code Stability [IMPLEMENTED]

30-bootstrap stability check (30 iterations × 500-row samples):

Feature	Top-5 presence	Stability tier
EXT_SOURCE_MEAN	30/30	HIGH
EXT_SOURCE_1	30/30	HIGH
CREDIT_TO_GOODS	30/30	HIGH
CREDIT_TO_ANNUITY	30/30	HIGH
INST_LATE_RATIO	30/30	HIGH

EXT_SOURCE_MEAN is rank-1 in all 30 bootstraps. The top-5 reason-code set is robust to sampling variation — suitable for production adverse-action drafting (subject to officer review).

Section 12 — Fairness / Proxy Audit [PARTIALLY VERIFIED]

Scope: Proxy-variable analysis on 61,503-row test set. No protected-class labels available in Home Credit.

Proxy Group	Approval Rate	Observed DR	Assessment
Age < 30	80.5%	11.5%	DR differential explains gap
Age 30–50	88.6%	8.8%	—
Age 50+	95.5%	5.6%	DR differential explains gap

Proxy Group	Approval Rate	Observed DR	Assessment
Income low tercile	89.0%	8.1%	Narrow spread
Income mid tercile	88.5%	8.8%	—
Income high tercile	91.3%	7.5%	—
Employed	88.3%	8.7%	Normal
Not-employed proxy	96.2%	5.2%	Sentinel encodes retired/etc. — lower DR
Region 1 (low-risk)	96.5%	4.9%	DR differential explains gap
Region 3 (high-risk)	81.7%	11.2%	DR differential explains gap

What this proves: No evidence of model amplifying approval-rate disparities beyond base-rate differences. Model appears to differentiate risk, not proxy demographics.

What it does not prove: Not a fairness certification. Disparate Impact ratio not computable (no protected-class labels). A full ECOA/fair-lending review would require demographic enrichment and independent analysis.

Section 13 — Drift / Vintage Stress `[PARTIALLY VERIFIED]`

Comparison	PSI	Interpretation
val vs test	0.0002	STABLE — key operational metric
train vs val	~8.10	Platt calibration artefact (not a drift signal)

The train PSI is explained by the Platt calibrator being fit on val-set scores. The calibrated val/test distribution is consistent (PSI=0.0002). All top-10 feature PSIs between train and test are below 0.001.

AUC by split: train=0.8526 (in-sample), val=0.7734, test=0.7769 — consistent val/test, expected train overfitting.

True vintage validation `[FUTURE]`: Requires timestamped application data. Home Credit has none. Monthly PSI monitoring on score distribution + top-10 features is the production monitoring recommendation.

Section 14 — Local RAG / LLM Governance Assistant `[SIMULATED]`

Architecture:

- Retriever: BM25 over 5-document policy corpus
- Abstain threshold: BM25 top-score < 0.25 → no output (prevents hallucinated policy citations)
- LLM role: Draft memo and adverse-action language from retrieved policy section
- Hard rule: LLM never autonomously approves or rejects applicant in any case
- All outputs: ASSISTIVE_ONLY + HUMAN_REVIEW_REQUIRED + NOT_FINAL_DECISION

6 cases demonstrated:

Case	Band	Retrieved Policy	LLM Output	Abstain
GREEN_APPROVE	GREEN	POL-001 (Score Band Policy)	Approval eligibility memo	No
AMBER_REVIEW	AMBER	POL-001	Review memo + top-3 SHAP drivers	No
RED_DECLINE	RED	POL-002 (Adverse Action Policy)	Adverse-action draft + 3 reason codes	No

Case	Band	Retrieved Policy	LLM Output	Abstain
CONFLICT_OVERRIDE	AMBER	POL-004 (Override Protocol)	Override checklist; officer authority noted	No
OUT_OF_DOMAIN	N/A	None (BM25=0.0)	Abstain — no output	Yes
DRIFT_ALERT	MONITOR	POL-003 (Monitoring Policy)	MRC escalation memo	No

What the LLM supports: Draft memos, reason-code language, policy lookups, checklist generation, escalation flagging.

What the LLM never does: Make or suggest credit approval/rejection, claim legal compliance, access live applicant data.

Section 15 — Evidence Artifacts

Artifact	Purpose	Status	Key Numbers	Claim Enabled
`gold_pass1_artifact_audit.json`	G9A artifact completeness	HIGH	23/23 PASS	"All G9A artifacts verified"
`gold_pass1_data_split_validation.json`	Split integrity	HIGH	21/21 PASS, 0 ID overlap	"No data leakage between splits"
`gold_pass1_leakage_audit.json`	Pre-tuning leakage	HIGH	10/10 PASS, safe_to_tune=true	"No target or post-outcome leakage"
`gold_pass1_temporal_vintage_feasibility.json`	Temporal analysis	HIGH	FEASIBILITY_LIMITED	"No temporal split possible; documented"
`gold_pass1_tournament_quality_audit.json`	Baseline metrics	HIGH	CatBoost AUC=0.7716, KS=0.4094	"12-model baseline with documented metrics"
`gold_pass1_rag_llm_governance_audit.json`	RAG audit	HIGH	10/10 PASS, abstain verified	"BM25 abstain functional"
`gold_pass2_tuning_trace.json`	HPO trace	HIGH	5 models, 4–8 trials, bug documented	"Validation-only Optuna HPO"
`gold_pass2_validation_model_comparison.json`	Tuned comparison	HIGH	LGB_mono=0.7734 #1	"Champion by composite score"
`gold_pass2_calibration_report.json`	Calibration	HIGH	Platt ECE_val=0.0051	"Platt calibration selected"
`gold_pass2_champion_selection_report.json`	Selection	HIGH	Composite=0.7312	"9-component composite champion"
`gold_pass2_final_untouched_test_report.json`	Test metrics	HIGH	AUC=0.7769, ECE=0.0034	"Final test: AUC=0.7769"
`gold_pass3_threshold_scoreband_report.json`	Score bands	HIGH	GREEN 89.7%, AMBER 9.8%, RED 0.5%	"Score-band policy"
`gold_pass3_cost_sensitive_decisioning.json`	Cost analysis	HIGH (simulated)	t=0.47, EL=0.0807	"Cost-sensitive decisioning (scenario)"
`gold_pass3_shap_reason_code_report.json`	SHAP	HIGH	EXT_SOURCE_MEAN=0.510, 4 local cases	"SHAP reason codes"

Artifact	Purpose	Status	Key Numbers	Claim Enabled
`gold_pass3_reason_code_stability.json`	Stability	HIGH	30/30 top-5, 30/30 rank-1	"Reason codes stable across bootstraps"
`gold_pass3_fairness_proxy_audit.json`	Fairness	HIGH (partial)	4 proxy groups, aligned with DR	"Fairness proxy audit skeleton"
`gold_pass3_drift_vintage_stress.json`	Drift	HIGH (partial)	PSI=0.0002, all features STABLE	"val-vs-test PSI=0.0002 STABLE"
`gold_pass3_rag_llm_demo_report.json`	RAG demo	HIGH (simulated)	6 cases, abstain fires	"RAG/LLM 6-case governance demo"
`pulseguard_final_gold_audit.json`	Final audit	HIGH	89.3%, GOLD	"PulseGuard is GOLD checkpointed"
`docs/PULSEGUARD_MODEL_CARD.md`	Model card	HIGH	All lifecycle fields	"Complete model card"
`docs/PULSEGUARD_GOLD_PASS3_GOVERNANCE_REPORT.md`	Governance	HIGH	Pass 3 verdict ACCEPT	"Governance report produced"
`docs/PULSEGUARD_GOLD_CHECKPOINT.md`	Checkpoint	HIGH	89.3%, GOLD	"Project frozen for defense"
`04_EVIDENCE_LEDGER.md`	Claim ledger	HIGH	All claims traced	"Every claim has an artifact"
`06_CLAIM_BOUNDARY.md`	Claim boundary	HIGH	Safe/forbidden per gate	"Claim boundary maintained"

Section 16 — Failure Modes

#	Failure	Why It Happens	Current Detection	Mitigation	Remaining Gap	Interview Framing
1	Target leakage	TARGET or post-outcome variable in features	10/10 pre-tuning leakage audit (GP1)	TARGET explicitly excluded; audit confirms	No automated re-audit if features change	"I ran a 10-check leakage audit before any tuning. safe_to_tune=true confirmed."
2	Test-set leakage	HPO uses test set for model selection	Calibrator fit on val only; test evaluated exactly once (GP1 check #7)	Optuna optimises on val AUC only	Manual discipline required for future additions	"Test set was touched exactly once, after all decisions were locked."
3	Post-outcome variables	Side-table features populated after loan outcome	Temporal leakagechecks; DAYS_DECISION in side-tables reviewed	GP1 GP1 confirmed no temporal proxies at cut	Future table additions require re-audit	"I checked bureau/installment side-tables for outcome-proxying features."

#	Failure	Why It Happens	Current Detection	Mitigation	Remaining Gap	Interview Framing
4	Temporal/vintage overclaim	Claiming out-of-time validation when split is random	GP1 temporal feasibility: FEA SIBILITY_LIMIT ED	Documented; no OOT claim made	True vintage requires timestamped data	"Home Credit has no timestamps. I cannot do out-of-time validation and I say so."
5	AUC-only champion selection	Selecting model with highest AUC regardless of governance	9-component composite score; governance premium for monotone constraints	Composite explicitly weights ECE, KS, explainability, adverse-ready	—	"I didn't select by AUC alone. CatBoost has similar AUC but no monotone constraints — it loses the composite."
6	Calibration overfit	Isotonic regression fits val perfectly (ECE=0)	Isotonic excluded from comparison; Platt selected	ECE_val documented; test ECE=0.0034 is the real measure	—	"Isotonic ECE=0.0 on val is an overfitting artifact. Platt is the correct calibrator."
7	SHAP instability	Global SHAP changes significantly across resamples	30-bootstrap stability check	Top-5 features all 30/30; rank-1 30/30	Sample estimate only; full-corpus SHAP would be stronger	"I bootstrapped SHAP 30 times. Top-5 features appear in all 30 runs."
8	Reason-code instability near threshold	AMBER applicants near PD=0.20 or PD=0.40 have unstable reason codes	Conflict case (AMBER_CONFLICT, PD=0.347) computed	Local case analysis; stability check covers range	Fine-grained near-boundary stability not measured	"Near-boundary cases are routed to manual review (AMBER band) — human officer adds context."
9	Fairness/proxy overclaim	Claiming fairness compliance from proxy analysis	SKELETON label on fairness report; forbidden claims documented	Proxy analysis only; no DI ratio claim	Protected-class labels not available	"Fairness is a skeleton — proxy analysis only. No protected-class labels means no DI ratio."
10	Cost matrix overclaim	Presenting scenario economics as real bank parameters	SCENARIO_ASSUMPTIONS_NOT_REAL_BANK_ECONOMICS label	Explicitly labelled; interview answer prepared	Requires real LGD/NIM data	"These are scenario assumptions. Real bank would need actual charge-off data and cost-of-funds."

#	Failure	Why It Happens	Current Detection	Mitigation	Remaining Gap	Interview Framing
11	Drift hidden by random split	PSI looks stable because val/test are from same random split	Documented as limitation; vintage analysis described as PARTIALLY_VERIFIED	val-vs-test PSI=0.0002 is correct metric; train PSI artefact explained	No true temporal drift test possible	"val-vs-test PSI is the meaningful stability metric. Train PSI is a calibration artefact."
12	RAG retrieves irrelevant policy	BM25 returns wrong document; LLM drafts from wrong policy	Abstain threshold (BM25<0.25 → no output)	OOD query (Case 5) abstains correctly	Small corpus may have weak coverage	"If BM25 score is below 0.25, the assistant abstains. That's the OOD safety net."
13	LLM makes unauthorized decision	LLM output framed as approve/reject	Hard rule: LLM never approves /rejects in any of 6 demo cases	ASSISTIVE_ONLY tag enforced throughout	Requires ongoing prompt discipline in production	"The LLM output in all 6 cases is a draft memo. It never says approve or reject."
14	Adverse-action legal overclaim	SHAP-drafted reason code presented as ECOA-compliant	Hard disclaimer on all SHAP outputs	ASSISTIVE_ONLY + HUMAN_REVIEW_REQUIRED	Licensed credit officer must review	"SHAP reason codes are draft language. They require officer review before any applicant communication."
15	Public-data-to-production overclaim	Claiming model is production-ready based on public data results	PORTFOLIO PROJECT clearly stated throughout	Model card explicitly: NOT_PRODUCTION_READY	Approved-applicant MNAR bias unaddressed	"This is a portfolio project on public data. Production would require reject inference, temporal validation, and independent model validation team review."

Section 17 — Implemented vs Future

Component	Status	Evidence	Safe Claim	Future Extension
Multi-table feature engineering	`[IMPLEMENTED]`	`g9a_feature_factory_report.json`	"140 features from 7 tables"	WOE/IV scorecard secondary model
Leakage audit	`[IMPLEMENTED]`	`gold_pass1_leakage_audit.json`	"10/10 PASS, safe_to_tune=true"	Re-audit after future feature additions
12-model baseline tournament	`[IMPLEMENTED]`	`g9a_model_tournament_report.json`	"12-model tournament, 2 hard-failures documented"	Neural baseline with GPU

Component	Status	Evidence	Safe Claim	Future Extension
Optuna HPO	`[IMPLEMENTED]`	`gold_pass2_tuning_trace.json`	"Validation-only Optuna TPE, 4–8 trials"	100-trial GPU search
Platt calibration	`[IMPLEMENTED]`	`gold_pass2_calibration_report.json`	"ECE=0.0034 post-Platt"	Beta calibration; group-specific calibration
Composite champion selection	`[IMPLEMENTED]`	`gold_pass2_champion_selection_report.json`	"9-component composite score"	DeLong test significance
Score-band policy	`[IMPLEMENTED]`	`gold_pass3_threshold_scoreband_report.json`	"GREEN/AMBER/RED PD-semantic thresholds"	Threshold calibrated to real LGD
SHAP reason codes	`[IMPLEMENTED]`	`gold_pass3_shap_reason_code_report.json`	"SHAP top-5 stable 30/30"	Full 61k-row SHAP computation
Fairness proxy audit	`[PARTIALLY VERIFIED]`	`gold_pass3_fairness_proxy_audit.json`	"Proxy audit; no amplification beyond DR"	Full DI analysis with protected-class labels
Drift/PSI baseline	`[PARTIALLY VERIFIED]`	`gold_pass3_drift_vintage_stress.json`	"val-vs-test PSI=0.0002 STABLE"	Monthly monitoring dashboard; OOT validation
RAG/LLM demo	`[SIMULATED]`	`gold_pass3_rag_llm_demo_report.json`	"6-case demo; abstain fires; ASSISTIVE_ONLY"	50+ doc corpus; live LLM integration
Champion/challenger monitoring	`[FUTURE]`	—	—	Challenger promotion framework
Production serving API	`[BUILT — demo model]`	`app.py` · Cloud Run endpoint	"FastAPI deployed to Cloud Run; serves G4 XGBoost demo (AUC=0.6261, 28 features, synthetic); GP2 champion blocked by sklearn 1.7.2/Python 3.9 pkl incompatibility"	Deploy champion after environment-parity fix (ONNX or matched base image)
Reject inference	`[FUTURE]`	—	—	Semi-supervised correction
Real temporal validation	`[FUTURE]`	—	—	Requires timestamped dataset

Section 18 — Claim Boundary

Locked Safe Claims

- "Tuned LightGBM_monotonic + Platt: AUC=0.7769, PR-AUC=0.2628, KS=0.4141, ECE=0.0034 on 61k held-out test"
- "Validation-only Optuna HPO; champion selected by 9-component composite score"
- "Monotone constraints on 15 features support SR 26-2 directional interpretability"
- "EXT_SOURCE_MEAN rank-1 SHAP driver in 30/30 bootstraps; top-5 features stable 30/30"
- "Score-band policy: GREEN<0.20 (89.7%, DR=5.8%), AMBER 0.20–0.40 (9.8%, DR=27%), RED≥0.40 (0.5%, DR=54%)"
- "RAG/LLM: abstains on OOD (BM25<0.25); LLM never makes credit decisions"

7. "Fairness proxy: approval-rate differentials aligned with DR differentials; no amplification"
8. "val-vs-test PSI=0.0002; all top-10 feature PSIs STABLE"
9. "PulseGuard scores GOLD at 89.3% (134/150) on 15-dimension governance audit"
10. "FastAPI scoring endpoint deployed to Cloud Run (<https://pulseguard-api-98058433335.us-central1.run.app>); serves G4 XGBoost demo model (AUC=0.6261, 28 features, 50k synthetic rows) — GP2 LightGBM champion NOT deployed due to sklearn 1.7.2/Python 3.9 pkl version incompatibility; serving gap fully documented"

Locked Forbidden Claims

1. Production lending system or live deployment
2. SR 26-2 certified or regulatory compliance certified
3. Legally compliant adverse action notice
4. Fairness certified or disparate impact compliant
5. Real bank data or real lender economics
6. Full vintage/out-of-time validation
7. LLM makes or influences credit decisions
8. 100 Optuna trials completed (actual: 4–8)
9. CatBoost is the final champion (champion is LightGBM_monotonic)
10. AUC=0.7716 is the tuned result (that is the BASELINE untuned CatBoost score)
11. "Live endpoint AUC=0.7769" — endpoint serves G4 XGBoost synthetic demo; AUC=0.6261 on synthetic data
12. "The champion model is deployed" — GP2 LightGBM champion pkl files are unloadable on Python 3.9; endpoint runs the G4 demo model

Weak Wording to Avoid

- "Production-grade" — say "portfolio project"
- "State-of-the-art" — say "competitive on Home Credit at AUC=0.7769"
- "Fair model" — say "fairness proxy audit; no evidence of amplification beyond base rates"
- "Drift-tested" — say "val-vs-test PSI=0.0002; true vintage validation not possible"
- "Compliant" — say "designed to be SR 26-2 aligned; not certified"

Resume-Safe Wording

"Built end-to-end credit-risk ML pipeline on Home Credit Default Risk (307k applicants, 57.4M rows); champion LightGBM + Platt achieves AUC=0.7769, KS=0.41, ECE=0.0034 — selected via 9-component composite score including monotone constraints, calibration, and adverse-reason readiness."

Interview-Safe Wording

"The champion is LightGBM with monotone constraints on 15 features, calibrated with Platt sigmoid. Final untouched test AUC is 0.7769. I selected it not by AUC alone but by a composite that includes ECE, KS, PR-AUC, latency, and governance criteria. The model is documented with a full model card, evidence ledger, and claim boundary."

Section 19 — Interview Q&A Master Deck

Q1 — 60-second opener: "Tell me about PulseGuard in one minute."

"PulseGuard is my credit-risk governance portfolio project built on the Home Credit Default Risk dataset — 307,000 applicants, 57 million rows across 7 relational tables. I engineered 140 features, ran a 12-model baseline tournament, then tuned 5 models with Optuna hyperparameter search. The champion is LightGBM with monotone constraints, calibrated with Platt sigmoid — AUC 0.77, ECE 0.003 on a 61,000-row holdout. I then added the governance layer: score-band policy, SHAP reason codes with bootstrap stability, a fairness proxy audit, a drift baseline, and a local RAG policy assistant that drafts adverse-action memos for credit officer review. Every claim is traceable to an evidence artifact. It's not a production system — it's a demonstration that I understand the full governed ML lifecycle."

Q2 — 2-minute walkthrough: "Walk me through the project."

"I'll cover it in four stages: data, model, governance, and boundaries.

>

Data. Home Credit Default Risk — 307,000 applicants at 8% default rate, across 7 tables including bureau history, instalment payments, credit cards, and POS cash records. I engineered 140 features: ratio features like credit-to-annuity and loan-to-goods-value, behavioural aggregates like late-payment ratio and bureau overdue ratio, and a composite behavioral risk score. I ran a 10-check leakage audit before any model training — the test set was never touched until the final evaluation.

>

Model. I ran a 12-model baseline tournament, identified that LightGBM was underperforming due to an early-stopping bug on imbalanced data, then ran Optuna hyperparameter search on 5 models. Champion: LightGBM with monotone constraints on 15 features — selected by a 9-component composite score covering AUC, calibration, KS, PR-AUC, latency, and governance criteria. Post-tuning Platt calibration produces ECE=0.003 on the test set. Final untouched test AUC: 0.7769.

>

Governance. I built the governance layer: a three-zone score-band policy (GREEN/AMBER/RED), SHAP reason codes stable across 30 bootstrap resamples, a fairness proxy audit on age/income/employment/region, a PSI drift baseline, and a local BM25 policy RAG with an LLM that drafts adverse-action memos — ASSISTIVE_ONLY, the LLM never makes credit decisions.

>

Boundaries. This is a portfolio project, not a production system. Limitations documented: no temporal validation possible (no timestamps), approved-applicant selection bias unaddressed, fairness audit is proxy-only, and cost matrix is scenario-assumed. Every claim in my resume has an artifact behind it."

Q3 — 5-minute deep dive: "How did you approach feature engineering?"

"I treated each side-table as a separate signal domain and engineered specific features per domain.

>

For the application table, I built ratio features that proxy lending concepts: CREDIT_TO_INCOME as a debt-to-income proxy, CREDIT_TO_GOODS as a loan-to-value proxy, CREDIT_TO_ANNUITY as a repayment capacity metric, and ANNUITY_TO_INCOME as a debt service ratio. I also built a flag for the Home Credit encoding quirk: DAYS_EMPLOYED=365,243 means 'not employed' — I exposed that as FLAG_EMPLOYED_ANOMALY.

>

For the bureau tables, I aggregated at the applicant level: count of active credit lines, overdue ratio, total overdue balance. The bureau balance table gave me mean DPD ratio across monthly snapshots — a leading indicator of credit stress.

>

For installments, I computed the late-payment ratio — INST_LATE_RATIO — which turned out to be the fourth-strongest SHAP driver globally.

>

I built a composite behavioral risk score — $0.4 \times \text{INST_LATE_RATIO}$ plus $0.3 \times \text{CC_DPD_RATIO}$ plus $0.2 \times \text{POS DPD ratio}$ plus $0.1 \times \text{bureau overdue ratio}$ — to capture the multivariate payment stress signal in a single defensible feature.

>

I then ran a leakage audit before any modeling. The 10-check audit confirmed TARGET is not in features, no post-outcome bureau fields leaked, and the val and test splits have zero applicant ID overlap with training."

Q4 — "Why Home Credit and not Taiwan or LendingClub?"

"Home Credit has 10× the sample size of Taiwan (307k vs 30k), 7 relational side-tables vs Taiwan's single table, an 8% default rate that matches real consumer-loan portfolios (Taiwan's 22% is credit card), and real external bureau signals as primary risk drivers. Taiwan was used at an earlier stage of the project as a realism bridge — it's retained for historical context only.

>

LendingClub is dropped. The main reason to use LendingClub would be for reject inference — studying the application population that was declined. That's an important workstream but it's a separate project, and building reject inference badly would produce misleading results. I documented it as a future build."

Q5 — "What did you build in terms of modeling?"

"Twelve baseline models: logistic regression, random forest, gradient boosting variants, CatBoost, XGBoost, LightGBM, and monotonic constraint variants. Two hard-failed: TabNet would take 400–800 hours on CPU, and sklearn GBM hit the wall-time limit — both documented with cause, not silently dropped.

>

Then Optuna hyperparameter search on five models: CatBoost, XGBoost, XGBoost with monotone constraints, LightGBM base, and LightGBM with monotone constraints. Validation-only — test set not touched. Champion selected by a 9-component composite score."

Q6 — "How did you avoid leakage?"

"Three layers. First, I excluded TARGET from the feature set — confirmed by the pre-tuning leakage audit, which checked TARGET correlation with every feature. Second, the Platt calibrator was fit on the validation set only — not on train, not on test. Third, the test set was used exactly once, after the champion was locked, calibration was complete, and the score-band thresholds were defined. The 10-check audit also verified no applicant ID overlap between splits and no post-outcome proxy variables in the side-tables."

Q7 — "How did you split the data?"

"Stratified random 60/20/20 split — 184,506 train, 61,502 val, 61,503 test — with seed=42, preserving the 8.07% default rate across all three splits. The dataset has no timestamps, so out-of-time splitting is not possible. I documented that as a limitation — it's not something I tried to hide or work around."

Q8 — "Why LightGBM_monotonic and not CatBoost?"

"CatBoost val AUC is 0.7708, LightGBM_mono is 0.7734 — only a 0.0026 gap on AUC. But the composite score is 0.6802 for CatBoost vs 0.7312 for LightGBM_mono. The composite includes an explainability component: LightGBM_mono gets a full score for having both SHAP and monotone constraints — CatBoost only gets partial credit for SHAP. And there's a 5% weight for adverse-reason readiness — monotone constraints make every prediction auditable without SHAP.

>

More importantly: LightGBM_mono is both the performance champion and the governance champion. There's no trade-off to defend. That's a better story than 'we traded 0.002 AUC for governance compliance.'"

Q9 — "Why monotonic constraints?"

"Monotone constraints enforce directional interpretability at every tree split. If BUREAU_OVERDUE_RATIO has a +1 constraint, then for any two applicants identical in all other features, the one with the higher overdue ratio will always receive a higher predicted default probability — guaranteed by the model's construction.

>

This matters for SR 26-2 model risk management. An auditor can verify the directional claim by inspection without needing a SHAP explanation. A credit officer can explain the model's logic to a regulator: 'more late payments means higher predicted risk' — without showing a SHAP waterfall chart.

>

The cost is some flexibility in the tree splits. In practice, LightGBM_mono AUC=0.7734 vs LightGBM_base AUC=0.7724 — the monotone version is actually slightly better on AUC here, probably because the constraints act as useful regularisation on the 140-feature space."

Q10 — "How did tuning work?"

"Optuna Tree-structured Parzen Estimator — a Bayesian search that models which hyperparameter configurations produce good validation AUC and proposes new configurations accordingly. I searched over n_estimators, learning rate, num_leaves, max_depth, subsample, colsample_bytree, and regularisation parameters. All tuning optimised on validation AUC — test set never consulted.

>

Trial counts: 4–8 per model, not 100. CPU-only sandbox with a 44-second bash execution limit. I'm honest about that: the tuning plan specified 100 trials, actual counts are documented in the tuning trace. The results are a reasonable local optimum, not a globally exhaustive search. On GPU with 100 trials I'd likely get 0.001–0.003 further AUC improvement."

Q11 — "Why small trial count?"

"CPU-only sandbox with a 44-second execution limit per bash call. Each LightGBM trial at 140 features takes 7–9 seconds. That allows 4–5 trials in a single call. I documented this constraint honestly rather than inflating the count. The plan specified 100 trials; I ran 4–8 per model and stated that clearly. Honest documentation of constraint is better than claiming a search budget you didn't run."

Q12 — "How did you use calibration?"

"Isotonic regression calibration — a piecewise-monotone interpolation fit on the validation-set raw scores. I evaluated both Platt (logistic regression) and isotonic; isotonic achieved lower test ECE (0.0034) and was selected.

>

One thing worth noting: the GP2 training report initially said 'Platt selected', but forensic inspection of the saved pkl (`cal[selected]`) confirmed isotonic was actually used. The calibrated probabilities matched isotonic interp exactly and diverged from the Platt sigmoid. This kind of ground-truth verification — reading the artifact rather than the report — is important when you're debugging a serving discrepancy.

>

For deployment, I extracted the isotonic calibration as two numpy arrays (the X and Y threshold lookups) and replicated it with `np.interp(raw_prob, iso_x, iso_y)`. This means zero sklearn dependency at serve time — no version lock, no import overhead. The calibrator runs as a two-array numpy operation."

Q13 — "What is ECE?"

"Expected Calibration Error. It measures how well the model's predicted probabilities match observed default rates. A calibrated model with PD=0.20 should produce ~20% defaults in the bucket of applicants near 0.20.

>

ECE is computed by binning predictions into 10 buckets, then averaging the absolute difference between mean predicted PD and observed default rate, weighted by bucket size. Our champion achieves ECE=0.0034 on the test set — the model's predicted probabilities are extremely close to the observed outcomes.

>

For comparison, the raw uncalibrated LightGBM has ECE=0.29 — the model is severely overconfident. Platt calibration reduces ECE by ~98%."

Q14 — "What is KS?"

"Kolmogorov-Smirnov statistic — the maximum separation between the cumulative distribution functions of predicted scores for defaulters and non-defaulters. KS=0.41 means there's a 41-percentage-point gap between the two CDFs at the point of maximum separation.

>

It's a useful single-number measure of discriminative power that doesn't depend on a specific threshold. For credit scoring, KS > 0.40 is conventionally considered strong. Our champion at KS=0.4141 on the test set is solidly in that range."

Q15 — "Why PR-AUC matters?"

"At an 8% default rate, the dataset is class-imbalanced. ROC-AUC can look good even if the model is mediocre at identifying defaulters — because most predictions are negative and ROC rewards true negatives.

>

PR-AUC (precision-recall area under curve) is more informative for imbalanced data because it focuses on performance on the minority class (defaults). It answers: at each recall level, what's the precision? PR-AUC=0.26 on an 8% base rate is actually quite good — a random classifier would give PR-AUC ≈ 0.08 (the base rate). Our champion achieves 3.2× the random baseline on PR-AUC."

Q16 — "What is score banding?"

"Score banding converts a continuous predicted probability into a discrete policy zone. GREEN (PD<0.20) is auto-approve-eligible; AMBER (PD 0.20–0.40) goes to manual credit officer review; RED (PD≥0.40) goes to enhanced underwriting.

>

The boundaries are set based on calibrated PD semantics: PD=0.20 means the model estimates a 20% default probability. The GREEN band observed default rate (5.77%) is materially below 20%, confirming the model is conservative in GREEN assignment. This is more defensible than an arbitrary percentile cut."

Q17 — "How did you choose thresholds?"

"Three methods evaluated. First, KS-optimal: the single threshold that maximises separation between defaulters and non-defaulters — this gives $t=0.0961$, approving 50.7% of applicants at 5.6% default rate. Too aggressive for a practical policy.

>

Second, cost-optimal under scenario economics: $C_{bad}=10$, $C_{reject}=1$ gives cost-optimal $t=0.47$ — but that's based on assumed, not real, bank economics.

>

Third, PD-semantic: I chose PD=0.20 and PD=0.40 as boundaries because they have an interpretable meaning — the model estimates 20% and 40% default probability at those thresholds. The observed default rates in each band (5.8% GREEN, 27% AMBER, 54% RED) validate the semantic. This is the most defensible choice for a governance-focused project."

Q18 — "How did you create reason codes?"

"Using LightGBM's built-in `pred_contrib=True` parameter — which is the LightGBM implementation of SHAP TreeExplainer values. For each prediction, it returns the marginal contribution of each feature to the log-odds of default. I then rank these contributions by absolute value and map the top drivers to human-readable language.

>

For example: EXT_SOURCE_MEAN contribution of +0.94 for an AMBER applicant maps to 'External credit bureau composite score below threshold.' INST_LATE_RATIO contribution of +0.45 maps to 'High proportion of late instalment payments.'

>

I deliberately used LightGBM's built-in implementation rather than the external SHAP library, because the external library had a pandas index compatibility issue with our non-default-indexed DataFrames. The native implementation is equivalent for tree models and avoids the dependency."

Q19 — "Are they legally compliant adverse action notices?"

"No. ECOA and Regulation B require adverse action notices with specific reason codes drawn from an approved list, reviewed by a licensed credit officer. Our SHAP-derived language is a draft aid — it helps a credit officer identify the right reason codes faster. The output is always tagged ASSISTIVE_ONLY and HUMAN_REVIEW_REQUIRED. A credit officer must review, confirm, and sign off on any adverse action notice sent to an applicant.

>

We explicitly do not claim legal compliance. That boundary is documented in the model card, the governance report, the claim boundary document, and the RAG/LLM system's hard rules."

Q20 — "How did you audit fairness?"

"I ran a proxy audit — the maximum achievable without protected-class labels. Home Credit doesn't contain race, sex, or national origin fields. I used age bands, income terciles, employment status proxy, and regional rating as approximate proxies.

>

For each group I computed the approval rate at the GREEN band threshold ($PD < 0.20$) and compared it to the observed default rate for that group. The finding: all approval-rate differences are directionally aligned with default-rate differences. The model appears to differentiate risk, not discriminate by demographics.

>

This is explicitly labelled a governance skeleton, not a fairness certification. To compute a proper disparate impact ratio, I'd need demographic labels — which this dataset doesn't have."

Q21 — "What are the limitations?"

"Five material ones. First, reject inference: model trained on approved applicants only — MNAR selection bias. The performance on the declined population is unknown. Second, no temporal holdout: no timestamps in Home Credit; val and test are from the same distribution. Third, single geography: Eastern European portfolio — generalisation to other markets is untested. Fourth, CPU trial count: 4–8 Optuna trials instead of 100; reasonable local optimum, not globally optimal. Fifth, fairness: proxy audit only

— protected-class labels unavailable."

Q22 — "What would production need?"

"Eight things at minimum. Independent model validation team review (SR 26-2). Out-of-time validation on recent vintages. Real bank economics for the cost matrix. Fairness audit with real demographic labels and legal review. Reject inference correction for the approved-applicant selection bias. FastAPI serving with training-serving parity check. Monthly PSI monitoring harness with automated WARN/ALERT escalation. Credit officer workflow integration for adverse-action notice review and sign-off."

Q23 — "What does the LLM do?"

"The LLM governance assistant has three functions: policy retrieval, memo drafting, and adverse-action language suggestion. It takes an anonymised case summary, retrieves the relevant policy section using BM25, and drafts a memo for the credit officer to review.

>

The BM25 abstain threshold (0.25) prevents the LLM from generating output when no relevant policy is found — the out-of-domain query ('mortgage interest rate comparison') in Case 5 produces zero output, not a hallucinated policy citation.

>

The LLM never generates a credit decision. In all six demo cases, every output is either a memo, a checklist, or a reason-code draft — all labelled ASSISTIVE_ONLY. The credit officer sees the LLM output alongside the model score and makes the final decision."

Q24 — "Why can't the LLM decide?"

"Several reasons. Legal: credit decisions trigger adverse-action notification requirements under ECOA. Regulatory: an LLM-made decision may not produce an auditable reason-code trail required by Regulation B. Governance: SR 26-2 requires human accountability for model-driven decisions — an LLM substituting for a credit officer removes that accountability. Practical: the LLM has no access to the applicant's full file, prior override history, or real-time bureau data.

>

The ASSISTIVE_ONLY design is the only defensible architecture for any LLM in a regulated credit decision workflow."

Q25 — "How would you monitor drift?"

"Monthly PSI on the score distribution and top-10 input features. Thresholds: PSI<0.10 is STABLE (no action), 0.10–0.25 is SLIGHT_SHIFT (flag for model owner review within 30 days), PSI>0.25 is SIGNIFICANT_SHIFT (mandatory escalation to Model Risk Committee within 5 business days).

>

Additionally, if monthly KS drops more than 0.05 from the deployment baseline (0.4141), that triggers an emergency model review regardless of PSI. And the approve-zone observed default rate: WARN at 2× baseline (10%), ALERT at 25%, CRITICAL at 35%."

Q26 — "What would you improve next?"

"Top three. First, GPU with 100 Optuna trials — I expect 0.001–0.003 AUC improvement and a more robust champion. Second, temporal validation if I can find a timestamped public credit dataset — HMDA has application dates and would allow true out-of-time testing. Third, a stronger fairness audit with demographic enrichment — finding or acquiring protected-class labels for Home Credit or a comparable dataset is the highest-governance-value next step."

Q27 — "How does this relate to real credit risk?"

"The Home Credit dataset is derived from a real Eastern European consumer lender. The feature space — external bureau scores, instalment payment ratios, credit utilisation, employment stability — maps directly to what real credit risk models use. The difference is that real models also have income verification, employment verification, and a longer behavioral history window.

>

The governance artifacts — model card, evidence ledger, claim boundary, adverse-action reason codes, SHAP stability, PSI monitoring policy — are the exact same artifacts a real credit risk team produces before model deployment. The methods are industry-standard. The data is public and simplified. The gap to production is in complexity and governance sign-off, not in methodology."

Q28 — "How would you explain this to a business stakeholder?"

"The model estimates how likely each loan applicant is to default. Applicants are sorted into three buckets: Green for low risk, Amber for medium risk, and Red for high risk. Green applicants can be approved quickly — they have a very low estimated default rate. Amber applicants go to a credit officer for manual review. Red applicants need extra scrutiny or are declined.

>

When we decline an applicant or send them to review, the system drafts the top reasons — for example, 'high proportion of late payments in prior credit history' — for the credit officer to review and confirm. The officer makes the final decision; the model just helps them work faster and more consistently."

Q29 — "What are the biggest weaknesses?"

"Honest answer: three. First, reject inference — the model only learned from applicants the previous system approved. Applicants who were declined don't appear in the training data. If the previous system was biased, our model inherits that bias. Second, no temporal validation — I cannot demonstrate the model holds up on newer applicants; the train and test sets are from the same time period. Third, fairness — I can only do proxy analysis; I can't compute whether the model has disparate impact on protected classes without demographic labels."

Q30A — "I tested your live endpoint. The AUC is 0.62 — that's not the 0.77 on your resume. What's going on?"

"Correct, and it's documented. The live endpoint at Cloud Run demonstrates the serving architecture: preprocessing pipeline, Platt-calibrated probability, SHAP top-3 reason codes, score banding, and the ASSISTIVE_ONLY response structure. The model inside is the G4 demo model — XGBoost trained on 50,000 rows of synthetic data, 28 features, AUC=0.6261.

>

The GP2 LightGBM champion — AUC=0.7769, 140 features, 307k real Home Credit applicants — hit a deployment blocker. The.pkl artifacts were serialized under scikit-learn 1.7.2, which requires Python 3.10+. My deployment environment runs Python 3.9, which maxes out at sklearn 1.6.1. Cross-version.pkl deserialization isn't supported.

>

The serving pattern is what the endpoint is meant to demonstrate. If you want to evaluate champion quality, the evidence artifacts in `outputs/evidence/` have every metric traceable to a specific gate. Section 27D in this document has the full explanation."

Q30B — "What does 99.6% Bayes-optimal efficiency mean? Can you explain the Bayes ceiling?"

"The Bayes ceiling is the maximum achievable AUC given the information in the dataset — it's the performance you'd get from the true underlying probability function, not from a specific model. You can estimate it from the data-generating process if you know it.

>

For the G4 synthetic dataset, the DGP has exactly 6 signal features in a logistic model. I estimated the Bayes ceiling at AUC=0.6261 by scoring the held-out test set using the true DGP probability (oracle prediction). The G4 champion achieves AUC=0.6237 on the same test set.

>

The efficiency ratio: $0.6237 / 0.6261 = 99.6\%$. That means the model is capturing essentially all the information available from those 6 features — early stopping at 9 trees is the right decision, not a sign the model is undertrained.

>

This calculation only applies to the synthetic lane where I know the DGP. For Home Credit, the true Bayes ceiling is unknown. The 0.7769 champion AUC is the best I achieved; the true ceiling could be higher if additional features or better engineering were applied."

Q30C — "You said LightGBM was underperforming in the tournament. Exactly what was the bug?"

"LightGBM's early stopping checks the monitored metric (AUC) every `early\_stopping\_rounds` iterations and halts training if the metric hasn't improved by more than the tolerance. With `scale\_pos\_weight=11.39` for class imbalance, the gradient landscape is steep at iteration 1 — the first tree pushes AUC from 0.5 to roughly 0.7 in a single step. Each subsequent tree adds a marginal gain of perhaps 0.001–0.002. If `early\_stopping\_rounds=50` and the tolerance is the default, training can halt at 2–9 trees — the model looks like it ran normally but is severely undertrained.

>

The result in the G9A tournament: LightGBM AUC=0.7203. That looked like a legitimate result for 5 points below CatBoost. I accepted it as BASELINE_NOT_TUNED at Gold Pass 1 without diagnosing the root cause.

>

Gold Pass 2 fix: remove early stopping from the Optuna objective entirely. Treat `n\_estimators` as a search parameter (200–1000 range). Let Optuna control the tree count. After this fix, LightGBM_monotonic reaches AUC=0.7734 and becomes the champion. The bug pattern appears in any imbalanced problem with high `scale\_pos\_weight` — production fraud models at 0.1% positive rate have this risk at much more extreme scale factors."

Q30D — "TabNet hard-failed. What would TabNet need to actually work on this problem?"

"Two things: GPU and time. TabNet is a deep learning architecture that uses sequential attention to select features at each step — it processes the table row by row rather than building decision trees. Each epoch on a 184k-row dataset with 140 features takes roughly 6 minutes on CPU. The tournament has a 44-second execution limit per bash call. Estimated time to train 100 epochs on CPU: 400–800 hours. Hard fail, documented.

>

On GPU: each epoch drops to roughly 15 seconds, making a 100-epoch run a 25-minute job. The model would then be competitive on tabular data and worth including in the comparison.

>

Expected performance: TabNet on Home Credit would likely land between 0.77–0.79 AUC based on published benchmarks on similar tabular datasets. The main advantage over LightGBM is native feature selection via attention weights, which provides a different explainability modality. The main disadvantage: no monotone constraint equivalent, higher complexity, and requires careful regularisation to avoid overfitting on moderate-sized tabular data."

Q30E — "There's no temporal split. What would you actually do in production for temporal validation?"

"Four approaches, in order of quality. First, strict time-based split: if the dataset had application dates, I'd split at a fixed cutoff — say, train on applications before month T, validate on months T to T+6, test on T+6 to T+12. This tests whether the model holds up on genuinely unseen future applicants, not just a random subsample of the same vintage.

>

Second, rolling-window validation: retrain monthly on a rolling 24-month window, evaluate on the next month's cohort. This gives an estimate of the model's true production drift curve.

>

Third, if I can't get timestamped data: find a companion dataset that does have timestamps — HMDA has application dates and is publicly available, though the label definition is different (loan approval, not default). I'd retrain on HMDA and demonstrate temporal validation methodology, then note the dataset limitation.

>

Fourth, vintage stress test: even without true dates, I can split by applicant ID range (if IDs are approximately sequential) or by a proxy vintage indicator. I checked this on Home Credit: SK_ID_CURR default rate range is only 0.003 across quintiles — the IDs are essentially random, not time-ordered. So this approach isn't valid here, and I documented that explicitly.

>

The honest answer for Home Credit: temporal validation is not possible on this dataset. I say so in every interview. The methodology is ready; the dataset doesn't support it."

Q30 — "What's your SQL and Python implementation depth?"

"Feature engineering was done in pandas with multi-table merge and groupby aggregations — the bureau_balance table alone has 27 million rows, so I used memory-efficient dtypes and groupby aggregations rather than row-by-row operations.

>

For the PSI computation I wrote a custom function (10-equal-frequency-bin PSI with epsilon smoothing) rather than using a library, because I needed to understand the exact implementation. SHAP I used LightGBM's native `pred\_contrib` rather than the external SHAP library to avoid a pandas index compatibility issue.

>

If you want to see specific code: the feature factory is in `scripts/g9a\_feature\_factory.py`, the tuning script is in `scripts/gold\_pass2\_optuna\_tuning.py`, and the SHAP/threshold analysis is in `outputs/data/pass3\_\*.pkl` intermediates."

Section 20 — Resume Bullets

Short Bullets (1 line each)

- Built end-to-end credit-risk governance pipeline (Home Credit, 307k applicants) · LightGBM + Platt · AUC=0.7769 · ECE=0.0034 on 61k holdout
- Engineered 140 features across 7 relational tables (bureau, instalment, POS, credit card); composite behavioral risk score; FOIR/LTV/DTI proxies
- Delivered SR 26-2-aligned governance stack: SHAP reason codes, fairness proxy audit, score-band policy, PSI drift baseline, local RAG/LLM governance assistant

Strong Bullets (2–3 lines each)

- **Credit-risk ML governance stack, Home Credit Default Risk:** Engineered 140 features from 7 relational tables (57.4M rows); ran 12-model baseline + Optuna-tuned 5-model tournament; champion LightGBM with monotone constraints achieves AUC=0.7769, ECE=0.0034 post-Platt on 61k held-out test set; selected by 9-component composite including calibration, explainability, and adverse-reason readiness — not AUC alone.
- **End-to-end leakage-audited pipeline:** Pre-tuning 10-check leakage audit (safe_to_tune=true) ensured TARGET exclusion, val-only calibration, and single test-set evaluation; 60/20/20 stratified split with zero entity overlap; test set evaluated exactly once; all results traceable to `outputs/evidence/` artifacts.
- **SHAP governance layer with bootstrap stability:** Computed reason codes via LightGBM `pred\_contrib` for 4 local applicant cases; bootstrapped global feature importance across 30 resamples (500 samples each) — top-5 features appear in top-5 of all 30 bootstraps; EXT_SOURCE_MEAN rank-1 in 30/30; adverse-action drafts tagged ASSISTIVE_ONLY, HUMAN_REVIEW_REQUIRED.

ML/Risk-Focused Bullets

- **Calibration and score-band policy:** Platt sigmoid calibration reduces ECE from 0.296 (raw) to 0.0034 (test) — enabling semantic threshold design: GREEN<0.20 (89.7% of test, DR=5.8%), AMBER 0.20–0.40 (9.8%, DR=27%), RED≥0.40 (0.5%, DR=54%); cost-sensitive decisioning across 3 scenario assumptions documented.
- **Imbalanced-data modeling:** Handled 8.07% default rate (scale_pos_weight=11.39); diagnosed LightGBM early-stopping bug with scale_pos_weight (fires at iteration=1); fixed by treating n_estimators as Optuna hyperparameter; AUC improved from 0.7203 (baseline, bug present) to 0.7734 (tuned, fix applied).

Governance / GenAI Bullets

- **Local RAG/LLM governance assistant:** BM25 policy retriever over 5-document corpus; abstain threshold (BM25<0.25) prevents hallucinated citations; 6-case demo (approve/review/decline/conflict/OOD/drift); LLM drafts ECOA-style adverse-action memos for credit officer review — ASSISTIVE_ONLY, HUMAN_REVIEW_REQUIRED, NOT_FINAL_DECISION enforced.
- **SR 26-2-aligned model governance:** Produced model card, governance report, evidence ledger (all claims traceable to artifact), claim-boundary document (safe/forbidden for every gate), fairness proxy audit skeleton, PSI drift baseline, and override protocol; champion is simultaneously performance and governance champion (monotone constraints + SHAP); project scored GOLD at 89.3% on 15-dimension governance audit.

Unsafe Bullets to Avoid

- "Built production credit scoring model" — not production
- "Model approved for lending" — not approved
- "AUC 0.85+ on credit data" — champion is 0.7769
- "100 Optuna trials completed" — actual: 4–8
- "Fair model across protected classes" — proxy audit only
- "CatBoost champion at AUC=0.7716" — wrong champion, wrong metric; that's the untuned baseline

Section 21 — Final Defense Checklist

- [x] I can explain the project in 60 seconds (Section 19, Q1)
 - [x] I know every metric: AUC=0.7769, PR-AUC=0.2628, KS=0.4141, Brier=0.0668, ECE=0.0034
 - [x] I can explain the champion choice (composite score, not AUC alone; monotone governance premium)
 - [x] I can explain limitations (reject inference, no temporal validation, proxy fairness, CPU trial count)
 - [x] I can explain why this is not production lending (portfolio project; no independent validation, no regulatory review)
 - [x] I can defend the LLM boundary (ASSISTIVE_ONLY; 6 cases show LLM never decides)
 - [x] I can defend fairness/proxy limits (no protected-class labels; proxy audit only; SKELETON labelled)
 - [x] I can answer SQL/Python implementation questions (pandas groupby, pred_contrib, custom PSI, Optuna)
 - [x] I know where every metric lives: `outputs/evidence/gold\_pass2\_final\_untouched\_test\_report.json`
 - [x] I know the forbidden claims and can avoid them under pressure
-
-

Section 22 — Adversarial Follow-Up Drill

These are the second-punch questions that fire immediately after the standard Q&As. Practice answering the base question, then fielding the follow-up without a pause.

Q3 follow-up — "You mentioned 57.4M rows. That's the raw input, right? What's the actual modeling matrix?"

"Correct. Each side table is aggregated to applicant level via GROUP BY SK_ID_CURR — bureau_balance from 27M monthly records down to a handful of ratios per applicant, same for installments and POS tables. The final modeling matrix is 307,511 rows × 140 features. 57.4M is the raw data volume processed during feature engineering, not the size of what goes into the model."

Q3 follow-up — "What did you do with nulls?"

"Median imputation for numerics, mode for categoricals, all fit on train only and applied to val and test. I also added explicit missingness indicator flags for features where missingness itself is a signal — for example, OWN_CAR_AGE is null when the applicant owns no car, which is informative. LightGBM handles missing values natively in splitting decisions, so for tree models the imputation is belt-and-suspenders rather than critical."

Q6 follow-up — "What's the difference between data leakage and target leakage specifically?"

"Target leakage is when a feature is causally downstream of the outcome — it's populated after, or because of, the default event. For example, a 'collection agency referral' flag is populated post-default; including it would let the model trivially predict default. Data leakage is broader — it includes test set contamination, temporal future data bleeding into training, and group leakage where the same entity appears in both train and test. Target leakage is a specific subtype: the feature encodes information that only exists after the label is known."

Q7 follow-up — "Why not k-fold cross-validation?"

"Three reasons. First, calibration discipline: Platt calibration must be fit on a fixed held-out set; k-fold complicates this because there's no single stable validation set. Second, test isolation: with k-fold, every sample eventually appears in a validation fold, which makes clean test-set isolation harder to enforce. Third, at 307K rows the variance on a single 20% val split is already low — k-fold adds computational cost without meaningfully reducing estimator variance. For a smaller dataset I'd use k-fold; here it's unnecessary."

Q7 follow-up — "How does stratified splitting work mechanically?"

"sklearn's StratifiedShuffleSplit bins samples by class label, then samples from each bin proportionally to preserve class frequencies. With an 8.07% default rate and 307,511 rows, train gets ~184K rows at ~8.07% DR, val gets ~61.5K at ~8.07%, test gets ~61.5K at ~8.07%. Without stratification, random sampling variance at 8% minority class could produce splits ranging from ~7.5% to ~8.5% DR, which matters for calibration — the calibrator sees a different class prior than the test evaluator."

Q11 follow-up — "Why TPE over random search?"

"Random search treats every trial as independent — it samples the hyperparameter space uniformly with no memory of prior results. TPE builds a probabilistic model after each trial: it fits two density models — $P(\text{hyperparams} \mid \text{good val AUC})$ and $P(\text{hyperparams} \mid \text{bad val AUC})$ — and proposes configurations where the ratio of good-to-bad density is highest. After even 3–4 trials, TPE starts exploiting regions that worked instead of exploring blindly. With a budget of 4–8 trials per model, sample efficiency matters; random search would be pure luck."

Q11 follow-up — "Why not grid search?"

"Grid search is exponential in the number of dimensions. With 6 hyperparameters and 5 values each that's 15,625 combinations — completely impractical on CPU. Even with 3 values each it's 729. Grid search also forces you to pre-specify the search space densely; TPE adapts to the space as it explores."

Q12 follow-up — "Why $C=1e6$ in LogisticRegression for Platt?"

" C is the inverse of regularization strength in sklearn. $C=1e6$ means effectively zero regularization — the sigmoid fits the calibration training data as closely as possible. We want the Platt calibrator to find the best slope and intercept on the validation scores without being penalised for having a large slope. The only thing preventing overfitting is the held-out test set evaluation — $ECE_test=0.0034$ confirms the calibrator generalises."

Q12 follow-up — "What's the difference between Brier score and ECE?"

"Brier score is mean squared error of probability predictions — $\text{mean}((p_i - y_i)^2)$ — a proper scoring rule that penalises both miscalibration and poor discrimination. ECE is calibration-specific: it bins predictions, computes the gap between mean predicted probability and observed default rate in each bin, and averages across bins. A model can have low Brier but high ECE (good discrimination, bad calibration) or low ECE but high Brier (well-calibrated but weak discrimination). We report both: $\text{Brier}=0.0668$ measures overall probabilistic accuracy; $\text{ECE}=0.0034$ confirms the predicted probabilities are reliable."

Q14 follow-up — "How is KS computed exactly?"

"Sort all test samples by predicted probability. For each threshold t , compute the cumulative fraction of actual defaults predicted above t (TPR) and the cumulative fraction of actual non-defaults predicted above t (FPR). $KS = \max \text{ over all } t \text{ of } |TPR(t) - FPR(t)|$. It's the maximum vertical distance between the two CDFs. $KS=0.4141$ means there exists a threshold where the true positive rate exceeds the false positive rate by 41.4 percentage points — the model's maximum discriminative separation."

Q14 follow-up — "What's the difference between AUC and KS conceptually?"

"AUC is the probability that a randomly chosen defaulter scores higher than a randomly chosen non-defaulter. It integrates across the entire ROC curve — a global discriminative measure. KS is the maximum single-point separation between the two cumulative score distributions — a local discriminative measure at the best threshold. KS is widely used in credit risk because lenders often care about performance at a specific operating point; AUC is better for comparing models across all thresholds."

Q18 follow-up — "If SHAP reason codes aren't legally compliant, why build them at all?"

"Two reasons. First, they're the most efficient way for a credit officer to identify the right adverse action reason codes from an approved list — they don't replace the officer's judgment, they direct attention to the top drivers so the review is faster and more consistent. Second, they're governance evidence: they demonstrate the model is explainable, that the top drivers are sensible and directionally correct, and that the reason-code set is stable across resamples. A model that can't produce coherent reason codes is harder to defend in a supervisory review regardless of AUC."

Q20 follow-up — "val-vs-test PSI is 0.0002. Is that too good? Does it suggest something is wrong?"

"No — it's expected. Val and test are from the same stratified random split of the same dataset, drawn from the same distribution at the same point in time. PSI close to zero is the correct result when there's no temporal shift. The interesting PSI question would be comparing a new vintage of applicants to the training distribution — which isn't possible here because Home Credit has no timestamps. If val-vs-test PSI were high, that would suggest a bug in the split, not a real drift signal."

Section 23 — Core ML Concept Drill

These questions come up because of PulseGuard but aren't about PulseGuard — they test whether you understand the tools you used.

Q31 — "How does LightGBM differ from XGBoost architecturally?"

"Two main differences. First, tree growth strategy: LightGBM uses leaf-wise (best-first) growth — at each step it splits whichever leaf produces the largest loss reduction, regardless of depth. XGBoost uses level-wise growth — it completes all splits at depth d before going to depth $d+1$. Leaf-wise converges faster and finds lower training loss, but can overfit on small datasets; at 184K training rows it's appropriate. Second, split-finding: LightGBM uses histogram-based binning — continuous features are bucketed into 256 bins, and splits are found on bin boundaries. XGBoost's default uses exact split finding (evaluates every unique value), which is slower. LightGBM also has GOSS (gradient-based one-side sampling — downsamples low-gradient instances) and EFB (exclusive feature bundling — combines sparse features that rarely co-occur). These make LightGBM faster but the core gradient boosting math is the same."

Q32 — "What does scale_pos_weight do mathematically?"

"In gradient boosting, each sample contributes gradients to the tree-building objective. `scale_pos_weight` multiplies the gradient and hessian of every positive-class (minority) sample by that factor before they're summed across the node. The effect is equivalent to repeating each positive sample `scale_pos_weight` times in the loss computation. At `scale_pos_weight=11.39` the model treats each defaulter as if it were 11.39 samples, making misclassifying a defaulter 11.39× more costly than misclassifying a non-defaulter during training. It doesn't change the output probabilities directly — it changes what the trees optimise toward. For calibrated probability output, Platt scaling is still needed afterwards."

Q33 — "What was the early stopping bug and why did it happen?"

"LightGBM's early stopping halts training when the monitored metric stops improving for `early_stopping_rounds` consecutive rounds. With `eval_metric='auc'` on heavily imbalanced data (`scale_pos_weight=11.39`), the first tree already pushes AUC above the baseline significantly — the metric gains are concentrated in the first few iterations. If `early_stopping_rounds` is set to, say, 50, and the AUC gain from iteration 2 onward is less than the default tolerance, training stops at iteration 1. The result is a 1-tree model: AUC=0.7203, which looks like a reasonable result but is severely undertrained. The fix: treat `n_estimators` as an Optuna hyperparameter (search range 200–1000) with no early stopping, so Optuna controls the budget rather than the metric tolerance."

Q34 — "How does Platt scaling work mathematically?"

"Platt scaling fits a logistic function $f(s) = 1 / (1 + \exp(As + B))$ where s is the raw model score (log-odds output) and A, B are learned parameters. You feed the validation set raw scores and true labels into a LogisticRegression with `C=1e6`, which finds A and B by maximum likelihood. The result is a monotone transformation of the score that maps it to a well-calibrated probability. The intuition: if the model's raw scores are linearly related to the log-odds of the true probability (which they approximately are for GBMs), then a logistic regression on the scores recovers the correct probabilities. It's a 2-parameter correction — sufficient for most GBM miscalibration."

Q35 — "What is the bias-variance tradeoff in your specific context?"

"With 140 features and 184K training rows, variance (overfitting) is the dominant risk. Train AUC=0.8526 vs val AUC=0.7734 — a 0.08 gap — is the variance signature. I managed it with: (1) monotone constraints, which reduce the effective model complexity by excluding invalid tree splits; (2) Optuna searching over `lambda` (L2 regularisation), `alpha` (L1), `min_child_samples`, and `subsample`; (3) not over-tuning — 4–8 trials produces a stable local optimum rather than a finely tuned overfit. The Platt calibrator is intentionally low-bias (`C=1e6`, no regularisation) because we want it to fit the validation calibration curve exactly — its variance risk is controlled by evaluating it on the held-out test set."

Q36 — "What's a proper scoring rule? Is AUC one?"

"A proper scoring rule is a loss function minimised by reporting the true conditional probability — it incentivises honest probability estimates. Brier score and log-loss are proper. AUC is not: you can maximise AUC with any monotone transformation of the true probabilities, including extreme ones like 0.001 and 0.999. AUC measures ranking accuracy, not calibration. This is why we don't select the champion by AUC alone — a model with AUC=0.78 and ECE=0.29 (uncalibrated) is less useful than AUC=0.77 and ECE=0.003 (calibrated) for threshold-based policy decisions."

Q37 — "Why does PR-AUC matter more than ROC-AUC at 8% default rate?"

"ROC-AUC includes true negatives in the FPR denominator. At 8% DR, there are 11.4 non-defaulters per defaulter. A model that correctly ranks the easy majority-class samples scores well on ROC even if it's mediocre on the minority class. PR-AUC only considers precision (among predicted positives, how many are actual defaulters) and recall (among actual defaulters, how many

did we catch). A random classifier at 8% DR has PR-AUC≈0.08; our champion at PR-AUC=0.2628 is 3.3× the random baseline. PR-AUC is the honest measure of how well the model actually identifies defaulters."

Q38 — "What's the difference between monotone constraints and feature selection?"

"Feature selection removes features from the model entirely. Monotone constraints keep a feature in the model but restrict the direction of its effect. A +1 constraint on BUREAU_OVERDUE_RATIO means the model can still use that feature for any tree split, but only splits that increase the predicted score as overdue ratio increases are permitted. The model can capture non-linearities and interactions in the constrained direction — it just can't produce a split where higher overdue ratio leads to lower predicted default probability, which would be economically nonsensical. This gives interpretability guarantees without sacrificing predictive information."

Section 24 — Implementation Probes ("Did You Write This?")

These are the questions interviewers ask when they want to know if you actually ran the code or just read about it.

Q39 — "What shape is the pred_contrib output for LightGBM?"

"Shape (n_samples, n_features + 1). The last column is the bias term — the model's base score (essentially the log-odds of the training prior). To get feature-level SHAP contributions, slice [:,-1]. The sum of all columns including bias equals the raw log-odds prediction for that sample. To verify: booster.predict(X, raw_score=True) should equal contrib.sum(axis=1) for all samples."

Q40 — "What was the exact root cause of the pandas index error in SHAP?"

"y_val in the saved pickle was a pandas Series with the original Home Credit row labels — integers like 289233, 95708, 302524 — because it was sliced from the full DataFrame without resetting the index. green_idx was a numpy array of 0-based positional integers (0, 1, 2, 3...) from np.where. When you do y_val[green_idx], pandas interprets this as label-based lookup — it looks for rows labelled 0, 1, 2, 3 in the index. Those labels don't exist in a Series with index starting at 289233 → KeyError. Fix: y_val.values converts to a plain numpy array; then y_val_np[green_idx] is positional indexing, which works correctly."

Q41 — "What's the exact PSI formula you implemented?"

"Ten equal-frequency bins computed from the reference distribution using np.percentile(ref, np.linspace(0, 100, 11)). The bin edges are nudged (bins[0] -= 1e-9, bins[-1] += 1e-9) to ensure boundary values fall inside. Then np.histogram(ref, bins) and np.histogram(curr, bins) give counts; divide by n for proportions. Epsilon smoothing: replace any proportion below 1e-9 with 1e-9 to avoid log(0). PSI = sum over bins of (curr_pct - ref_pct) × log(curr_pct / ref_pct)."

Q42 — "How did you compute scale_pos_weight?"

"(1 - default_rate) / default_rate = (1 - 0.0807) / 0.0807 ≈ 11.39. This is the ratio of negative to positive samples in the training set. Equivalently: y_train.value_counts()[0] / y_train.value_counts()[1]."

Q43 — "What does stratify=y do in train_test_split exactly?"

"It uses StratifiedShuffleSplit internally, which bins samples by class label and samples from each bin proportionally. The result is that each split — train, val, test — gets the same approximate class ratio as the full dataset (8.07% defaults). Without it, at 8% DR a random split could swing ±0.5pp in default rate across splits just by chance, which would slightly bias calibration."

Q44 — "Why did you use LightGBM's native pred_contrib instead of the SHAP library?"

"Two reasons. Practically: the SHAP library's post-processing layer adds a pandas dependency that was incompatible with our non-default-indexed DataFrames — the exact root cause of the bug we chased across three sessions. Architecturally: for LightGBM tree models, the SHAP library calls LightGBM's native TreeSHAP implementation anyway — it's a wrapper, not a different algorithm. Using pred_contrib directly gives identical values with fewer dependencies. The only thing we lose is the SHAP library's summary_plot visualisation tools, which we didn't need for the JSON evidence artifacts."

Q45 — "If you called booster.predict(X, pred_contrib=True) on a 2-class problem with 50 features, what are the dimensions of the output?"

"(n_samples, 51) — 50 feature contribution columns plus 1 bias column. The bias is the model's expected output in the absence of any feature information (roughly equivalent to the log-odds of the base rate). For binary classification LightGBM returns a single output column per sample, not two (one per class) — it's the log-odds of the positive class."

Q46 — "Walk me through what happens when Optuna runs a trial."

"Optuna's TPE sampler proposes a hyperparameter configuration. The objective function receives it, builds the model with those parameters, trains on the training set, evaluates on the validation set, and returns the val AUC. Optuna logs the (hyperparams, val_AUC) pair. After ~5 trials, TPE fits two KDE models — one over the configurations that produced the top-25% of val AUC scores, one over the rest — and proposes the next configuration by maximising the ratio of the good-region density to the bad-region density. The `study.best_params` attribute gives the hyperparams of the best trial at any point. We set direction='maximize' since we're optimising val AUC."`

Section 25 — Behavioral / STAR Questions

These probe whether you can frame the project as professional experience, not just a technical exercise.

Q47 — "Tell me about a bug you hit and how you diagnosed it."

"The SHAP pandas index error is the best example. The symptom was a `KeyError` on integer indices — the kind of error that looks like a simple off-by-one but turned out to be a dtype mismatch. I'd tried three different approaches across separate sessions: list conversion, `.iloc``, explicit numpy casting. None fixed it because I was treating the symptom, not the cause.

>

The diagnosis came from printing the actual index of `y_val``: `index[:5] = [289233, 95708, 302524, 234022, 275284]`. That's not a 0-based index — it's the original Home Credit applicant IDs. `green\_idx` contained values like [1, 2, 3, 4, 5] — 0-based positional integers. Pandas label-based lookup for label 1 in a Series indexed at 289233 → not found → KeyError. Fix was one line: y_val.values`.`

>

The lesson: when you're debugging a pandas indexing error, the first thing to print is `.index``, not the values. The dtype and index type are separate — a Series of integers can have a non-integer or non-sequential index."

Q48 — "Tell me about a design decision you'd change."

"Using the SHAP library at all for tree models. The external SHAP library is valuable for model-agnostic explanations or for SHAP visualisation tools. For a LightGBM model, `pred\_contrib` is the native equivalent — same values, no dependency overhead. I planned to use the SHAP library because it's the standard portfolio recommendation, not because it added anything LightGBM couldn't do natively. If I'd read the LightGBM documentation on pred_contrib` first, I'd have used it from the start and avoided the index bug entirely."`

Q49 — "What's the most governance-conscious decision you made and why?"

"Selecting the champion by composite score rather than AUC alone. After tuning, `LightGBM\_monotonic` val AUC=0.7734 and CatBoost` val AUC=0.7708 — a gap of 0.0026. On AUC alone, that's barely a real difference. But the composite includes calibration (ECE), KS, PR-AUC, and a governance premium for monotone constraints and adverse-reason readiness. LightGBM\_monotonic` scores 0.7312 composite vs CatBoost` 0.6802.`

>

The governance-conscious part: by building the composite before running tuning, I committed to the selection criteria upfront — I couldn't retroactively choose whatever metric made my preferred model win. That's the same discipline a real model risk committee would expect: the champion selection criteria are documented before the models are evaluated."

Q50 — "What's the hardest limitation to defend?"

"Reject inference. The model was trained on approved applicants only — Home Credit never observed outcomes for applicants who were declined. If the previous lending policy systematically declined a particular demographic, the model inherits that bias: it has never seen a defaulter or non-defaulter from that group and its predictions for them are extrapolation, not interpolation.

>

There's no clean fix on this dataset. Semi-supervised reject inference methods exist — augmenting the training data with imputed outcomes for declined applicants — but they require strong assumptions about the declined population. I documented it as a material limitation in the model card. The honest interview answer is: this is a real problem in any model trained on bank data, PulseGuard doesn't solve it, and solving it properly would require either a randomised lending trial or a semi-supervised inference approach neither of which is possible on this public dataset."

Q51 — "What would you do differently with more time / GPU?"

"Three things, in order of impact. First: 100 Optuna trials with proper GPU compute. The current 4–8 trials per model is a reasonable local optimum, but the search space for LightGBM alone has at least 6–8 relevant hyperparameters. 100 trials would give TPE enough data to map the space well — I'd expect maybe 0.002–0.004 additional AUC and potentially better regularisation parameters. Second: find a timestamped public credit dataset — HMDA has application dates and approved/denied outcomes, which would let me build a true out-of-time split and run genuine vintage analysis. That's the single biggest methodological gap. Third: full fairness audit with demographic enrichment — the proxy analysis is the limit of what Home Credit allows; I'd want to replicate on a demographically labelled dataset to compute an actual disparate impact ratio."

Q52 — "How would you explain this project to a non-technical hiring manager?"

"Credit scoring is how a bank decides whether to approve a loan. The traditional approach is a spreadsheet of rules — if income is above X and debt is below Y, approve. PulseGuard replaces that with a machine learning model that learns from 307,000 past loan applications, each labelled as repaid or defaulted."

>

The model produces a probability — 'this applicant has a 12% chance of defaulting.' Below 20% we'd approve automatically. Between 20% and 40% a credit officer reviews it. Above 40% we decline or escalate."

>

What makes it a governance project rather than just a model is everything around that probability: documenting why the model makes each decision, checking it doesn't unfairly disadvantage any group, monitoring whether the predictions drift over time, and building a policy assistant that helps the credit officer draft the decline letter. All of that is what banks mean when they say 'model risk management.'"

Section 26 — Updated Defense Checklist (V2)

- [x] 60-second opener ready (Section 19, Q1)
- [x] 2-minute walkthrough ready (Section 19, Q2)
- [x] Every metric memorised: AUC=0.7769, PR-AUC=0.2628, KS=0.4141, Brier=0.0668, ECE=0.0034
- [x] Can explain champion selection (composite, not AUC; monotone governance premium)
- [x] Can explain 57.4M rows correctly (raw data volume, aggregated to 307K modeling grain)
- [x] Can explain limitations (reject inference, no temporal, proxy fairness, CPU trial count)
- [x] Can explain why not production (portfolio project; no independent validation)
- [x] Can defend LLM boundary (ASSISTIVE_ONLY; 6 cases; never decides)
- [x] Can defend fairness limits (no protected-class labels; proxy audit only)
- [x] Can answer adversarial follow-ups (k-fold, stratify, PSI=0.0002, SHAP legality)
- [x] Can answer core ML concepts (LightGBM vs XGBoost, TPE, bias-variance, proper scoring rules)
- [x] Can answer implementation probes (pred_contrib shape, index bug root cause, PSI formula)
- [x] Can answer behavioral questions (bug story, design decision, champion selection rationale)
- [x] Know the forbidden claims and avoid them under pressure

Section 27 — Role Expansion: Fraud Detection, MLOps, Risk Scoring

PulseGuard is frozen. No new code, no new data. This section reframes what already exists for three adjacent role contexts. The methodology doesn't change — the vocabulary does."

27A — Fraud Detection Roles

The honest bridge:

"PulseGuard solves imbalanced binary classification — 8.07% positive rate, scale_pos_weight=11.39, gradient boosting, Platt calibration, SHAP reason codes. Fraud detection is the same problem class at a lower positive rate (0.1–5%). The methods are identical. The feature domain (credit bureau history vs. transaction velocity features) and the regulatory vocabulary (ECOA adverse action vs. AML/fraud case narrative) differ. I'd frame PulseGuard to a fraud interviewer as: same toolbox, applied to credit risk, fully transferable."

Component mapping — what to say when asked:

Question	Fraud-context answer
"Have you worked on fraud detection?"	"Not directly — PulseGuard is credit default prediction. The problem class is identical: rare positive label, gradient boosting, calibration, SHAP explainability, PSI drift monitoring. The methodology maps 1:1; I'd be applying it to transaction or application fraud features rather than bureau aggregates."
"How would you handle class imbalance in a fraud context?"	"Same approach: scale_pos_weight (or class_weight='balanced') to upweight the minority class during gradient computation. At 8% default rate I used scale_pos_weight=11.39. For 0.5% fraud rate that'd be ~199. Platt calibration afterwards to recover well-calibrated probabilities — the raw GBM score overestimates positives with extreme class weighting."
"How would your drift monitoring apply to fraud?"	"PSI on the score distribution plus top-k feature PSIs. Fraud pattern drift is faster than credit drift — fraud rings adapt to models. I'd tighten the PSI WARN threshold (0.05 instead of 0.10) and add velocity-feature PSI checks. The monitoring policy structure is the same: STABLE / SLIGHT_SHIFT / SIGNIFICANT_SHIFT with SLA escalation times."
"How would SHAP reason codes work for fraud?"	"Same pred_contrib output from LightGBM. Instead of 'External credit bureau score below threshold', the top SHAP driver might be 'Unusual transaction velocity in last 24 hours'. The adverse-action framing becomes case-analyst language: the top 3 SHAP features get mapped to a fraud indicator narrative for the review queue. Still ASSISTIVE_ONLY — analyst confirms before any customer contact."
"What's the difference between credit and fraud model governance?"	"Credit: ECOA-driven adverse action requirement, SR 26-2 model risk management, long decision horizon (3–36 months to outcome). Fraud: faster outcome feedback (hours to days), higher velocity of model updates, AML/BSA regulatory context, real-time serving requirements. PulseGuard covers the governance methodology for both; the production infrastructure (latency SLAs, real-time feature stores) is a future build."

Safe fraud-context resume line:

"Methodology directly applicable to fraud detection: imbalanced binary classification (8% positive rate), LightGBM with monotone constraints, Platt calibration (ECE=0.0034), SHAP reason codes stable across 30 bootstrap resamples, PSI/KS drift monitoring — same algorithmic stack as fraud scoring systems, applied to credit default prediction."

What not to claim:

- "I built a fraud detection model" — false; data is credit default
- "I have fraud feature engineering experience" — no transaction velocity, device fingerprint, or network graph features
- "Real-time fraud scoring" — offline batch evaluation only

27B — MLOps / ML Platform Roles

The honest bridge:

"PulseGuard is a manually-implemented version of what an MLOps platform automates. Champion selection criteria → what Model Registry encodes. Evidence ledger → what MLflow experiment tracking stores. PSI/KS drift policy → what a monitoring dashboard surfaces. Model card + governance report → what a model review system produces. I built this by hand to understand what the automation is actually doing — which is exactly the background an MLOps platform engineer needs."

Component mapping — what to say:

Question	MLOps-context answer
"Walk me through your model lifecycle experience."	"End to end: data audit → leakage-checked feature engineering → baseline tournament → validation-only Optuna HPO → Platt calibration → 9-component composite champion selection → score-band policy → SHAP explainability → PSI drift baseline → model card + evidence ledger. The full governed lifecycle, implemented manually. In an MLOps platform role I'd be automating these steps — I understand every layer because I built each one."
"What's your experience with model monitoring?"	"PSI/KS drift monitoring with thresholds: PSI<0.10 STABLE, 0.10–0.25 SLIGHT_SHIFT, >0.25 SIGNIFICANT_SHIFT. WARN triggers model owner review within 30 days; ALERT triggers MRC escalation within 5 days. I implemented this as a Python function; in production it'd be a scheduled job feeding a dashboard. val-vs-test PSI=0.0002 confirmed score stability across the data split."
"Have you used MLflow or a model registry?"	"Not MLflow specifically — PulseGuard's evidence ledger is a manual equivalent. Every experiment result (trial count, val AUC, calibration ECE, composite score) is logged to a JSON artifact with the champion selection decision documented. The next build is MLflow integration — I know exactly what it'd need to store because I've been doing it by hand."
"What's a champion/challenger framework?"	"You deploy a champion model in production and route a fraction of traffic to a challenger. If the challenger meets your promotion criteria — higher composite score, statistically significant AUC improvement (DeLong test), no calibration regression — you promote it. PulseGuard has the 9-component composite scoring for this decision; the production A/B routing is a future build. I've designed the criteria; I haven't built the serving infrastructure."
"How would you handle a failing model in production?"	"Drift monitoring cascade: PSI/KS WARN triggers a model owner review. If the approve-zone observed default rate exceeds 2× baseline, that's a CRITICAL regardless of PSI. Immediate actions: freeze threshold changes, convene MRC, evaluate challenger. If no challenger is ready: revert to previous champion or apply a threshold adjustment while retraining. All of this is documented in PulseGuard's monitoring policy — `docs/G8_MONITORING_AND_INCIDENT_POLICY.md`."

Safe MLOps-context resume line:

"Implemented the governance layer of an ML platform by hand: 9-component composite champion promotion, evidence-traced artifact ledger (equivalent to MLflow experiment tracking), PSI/KS drift policy with WARN/ALERT/CRITICAL thresholds, model card, and claim boundary documentation — designed to demonstrate the mechanics that MLOps tooling automates."

What not to claim:

- "I built an MLOps platform" — governance documentation stack; no serving infrastructure
- "CI/CD for ML" — not implemented
- "Production model registry" — manual evidence ledger; no MLflow
- "FastAPI serving" — planned future build; not deployed

27C — Risk Scoring / Decision Science Roles

The honest bridge:

"PulseGuard covers the full scoring-to-decision chain: calibrated probability output, threshold economics, score-band policy design, reason-code explainability, and governance documentation. Decision science in fintech is exactly this — translating a model score into a policy with business consequences. PulseGuard demonstrates that translation explicitly."

Key talking points:

- Score-band thresholds are not arbitrary — they're set at PD=0.20 and PD=0.40 because the calibrated model's probabilities are interpretable. A business stakeholder can reason about "20% chance of default" without needing to understand the model.
- Cost-sensitive decisioning: documented scenario analysis under $C_{\text{bad}}=10$, $C_{\text{reject}}=1$. Cost-optimal threshold computed analytically ($\theta^* = C_{\text{reject}} / (C_{\text{bad}} + C_{\text{reject}})$). Sensitivity across 4 cost ratios. The framework is ready for real bank economics; the inputs are scenario assumptions.
- Champion selection by composite score is a decision science discipline: defining the criteria before running the experiment, not choosing the metric that makes the preferred model win.

27D — Live Endpoint Deployment Caveat

This subsection exists because interviewers may test the live URL and ask why the AUC doesn't match the champion metrics.

What's deployed vs. what the champion is:

	Live endpoint	PulseGuard champion
Model	G4 XGBoost + Platt	GP2 LightGBM_monotonic + Platt
Training data	synthetic_home_credit_like	Home Credit Default Risk
Rows	50,000	307,511
Features	28	140
Test AUC	0.6261	0.7769
URL	<code>`https://pulseguard-api-98058433335.us-central1.run.app`</code>	Not deployed

Root cause: GP2 pkl artifacts were serialized with scikit-learn 1.7.2 (requires Python 3.10+). The deployment machine runs Python 3.9, which maxes out at sklearn 1.6.1. Cross-version pkl deserialization is not supported — the champion artifacts are permanently unloadable on Python 3.9 without retraining.

Secondary issues hit during deployment:

- ``app.py`` initially loaded artifacts with ``pickle.load()``; ``train_champion.py`` saves with ``joblib.dump()``. Mixing the two raises ``_pickle.UnpicklingError: STACK_GLOBAL requires str``. Fix: ``joblib.load()`` throughout.
- ``src/feature_pipeline.py`` used ``ColumnTransformer | None`` union type hint (Python 3.10+ syntax). Fix: ``from __future__ import annotations`` at top of file (PEP 563 — treats all annotations as strings at runtime).

Q — "I tested your endpoint. The AUC is only 0.62 — that's not what your resume says."

"That's right, and it's documented. The endpoint demonstrates the serving architecture — preprocessing pipeline, Platt-calibrated probability, SHAP reason codes, score banding, ASSISTIVE_ONLY response structure. The model inside is the G4 demo model: XGBoost, 28 features, 50k rows of synthetic data, AUC=0.6261. The GP2 LightGBM champion — AUC=0.7769, 140 features, 307k real Home Credit applicants — hit a deployment blocker: the pkl artifacts were serialized with sklearn 1.7.2, which requires

Python 3.10+, and my deployment environment runs Python 3.9. The serving pattern is what the endpoint is meant to demonstrate. If you want to evaluate the champion, the evidence artifacts are in `outputs/evidence/` — every metric is traceable."

Q — "Why didn't you just retrain the champion on Python 3.9?"

"The hard constraint is: don't retrain the champion. The champion was selected by a 9-component composite score after 4–8 Optuna trials per model — retraining on Python 3.9/sklearn 1.6.1 would produce a different artifact, and I'd need to re-run the full validation chain to ensure it's the same model. That's reopening the gold pass, which is frozen. The honest answer is: I hit a version-pinning constraint late in the process. The right fix in a real environment is to pin sklearn at training time and match it at serving time. I documented it rather than patching it quietly."

Q — "What would you fix if you had more time?"

"Pin the sklearn version at G4 training time to match the deployment environment. Pin in `requirements.txt` at training, match in `Dockerfile`. The issue wouldn't have surfaced if training and serving had shared a `pyproject.toml` or a locked `requirements.txt` from day one. Alternatively: use ONNX export to decouple the model artifact from sklearn version entirely — an ONNX-serialized LightGBM is version-agnostic at inference time."

Section 28 — Failures I'm Proud Of

These are not polished success stories. They are genuine failures, diagnosed and fixed. Senior engineers learn from them. Interviewers respect them more than a narrative of unbroken success.

28.1 — DGP Default Rate Overshoot: G3.1

The failure: G3 was accepted with a synthetic DGP producing 26% default rate. Target was 8%. G4 was blocked.

What I had to do: Binary search on the logistic intercept. N=200,000 rows to reduce sampling noise. Converged at intercept -4.20703 in 8 iterations. Verified across 5 random seeds before proceeding.

The lesson: Synthetic data generation calibration is a real engineering task, not a throwaway step. The intercept shift from -2.8 to -4.2 moves the implied base rate from 26% to 8% — that's not obvious from inspection. Measure the output distribution before trusting the generator.

Interview framing:

"G3.1 is a good example of catching a dataset problem before it poisoned the model. A 26% default rate on a 8%-target dataset wouldn't have caused the model to fail — it would have caused it to train on the wrong problem silently. The model would have been calibrated for a much richer signal environment than the real one."

28.2 — LightGBM Early Stopping Bug: AUC=0.7203 Baseline

The failure: LightGBM's Gold Pass 1 baseline score was AUC=0.7203 — 5 points below CatBoost. Initial read: "LightGBM is worse on this dataset." The real cause wasn't diagnosed until Gold Pass 2.

What actually happened: With `eval\_metric='auc'` and `early\_stopping\_rounds=50` on scale_pos_weight=11.39 imbalanced data, the first tree dominates the AUC improvement. Subsequent trees have sub-threshold marginal gains. Training halts at 1–9 trees. The model looks like it ran normally but was severely undertrained.

What I had to do: Treat `n\_estimators` as an Optuna search parameter (200–1000 range). Remove early stopping from the Optuna objective entirely. After this fix, LightGBM_monotonic reached AUC=0.7734 and became the Gold Pass 2 champion.

The lesson: Early stopping parameters interact with class imbalance in non-obvious ways. In production fraud detection at 0.1% positive rate, the same early stopping risk exists at scale_pos_weight=999 — the first tree nearly saturates AUC improvement and stopping fires immediately. This is a real footgun that silently degrades fraud models.

Interview framing:

"The most important thing about this bug is that it looks like a correct result. AUC=0.72 is a plausible score. Without the Gold Pass 1 audit, I might have published it as the LightGBM result, documented LightGBM as inferior, and missed that the champion was sitting one hyperparameter fix away."

28.3 — SHAP Pandas Index Error: Three Sessions to a One-Line Fix

The failure: `y_val[green_idx]` raised `KeyError` on integer indices. Three debugging sessions, three different approaches (list conversion, `.iloc`, numpy casting), none successful.

The root cause: `y_val` had original Home Credit applicant IDs as its index (289233, 95708, ...). `green_idx` was 0-based positional integers from `np.where`. Pandas label lookup for `y_val[1]` searched for the row labelled 1 — which doesn't exist in a Series indexed at 289233.

The fix: `y_val.values`. One line. Three sessions to get there.

The lesson: When debugging pandas indexing, print `.index` before anything else. The values and the index dtype are orthogonal — a Series of integers can have a non-integer or non-sequential index. The symptom (wrong integer key) is identical for a type mismatch and a label mismatch.

Interview framing:

"This is the bug I'd use as a screening question for pandas expertise. 'You have a Series, you index it with an integer, you get a KeyError. What do you check?' The right answer is `.index`, not the values. Most people debug the values first and waste 45 minutes."

28.4 — Python 3.9 Union Type Annotation Crash

The failure: `src/feature_pipeline.py` used `ColumnTransformer | None` type annotation. On Python 3.9, this crashes at import with `TypeError: unsupported operand type(s) for |: 'ABCMeta' and 'NoneType'`. No warning at write time, no editor lint, crashes only at runtime on the target Python version.

The root cause: PEP 604 (`X | Y` union syntax) was introduced in Python 3.10. Python 3.9 only supports `Union[X, Y]` from `typing`. The `|` operator was only extended to type objects in 3.10 — in 3.9, it's a bitwise OR applied to the class metaclass and fails at runtime.

The fix: `from __future__ import annotations` (PEP 563) at the top of the file. This makes Python 3.9 treat all annotations as lazily-evaluated strings, not live objects. The `|` is never evaluated at runtime — the annotation is just a string. No refactoring of the annotation itself required.

The lesson: PEP 563 (`from __future__ import annotations`) is the forward-compatible fix for any Python version that doesn't support the annotation syntax you want to use. Writing `Union[X, Y]` is the other option but requires `from typing import Union`. The `__future__` import is cleaner when you want the annotation to be descriptive and not a runtime object.

Interview framing:

"This is exactly the kind of environment parity issue that slips through in portfolio projects but causes production incidents when a team upgrades Python versions. The `from __future__ import annotations` import is worth knowing because it's the one-line fix that makes modern annotation syntax backwards-compatible to Python 3.7+."

28.5 — sklearn 1.7.2 / Python 3.9 Pkl Version Lock-Out

The failure: GP2 LightGBM champion pkl artifacts serialized with sklearn 1.7.2 are permanently unloadable on Python 3.9/sklearn 1.6.1. `_pickle.UnpicklingError: invalid load key`. The champion model cannot be deployed to the production environment.

The root cause: sklearn pkl files embed internal class references. Cross-version deserialization is not guaranteed when the sklearn version changes. This is documented sklearn behavior, but it's easy to miss if training and serving environments aren't locked to the same version from day one.

What was done: Retrain G4 XGBoost from scratch under Python 3.9/sklearn 1.6.1. Deploy that instead. Document the serving gap fully. The champion remains the offline evaluation artifact; the endpoint serves the demo model.

The lesson: Version-pin your training environment and match it at serving time. The correct practice: shared `pyproject.toml` or locked `requirements.txt`, training and serving containers built from the same base image, or ONNX export to make the model artifact version-agnostic.

Interview framing:

"If I were building this in a production team, this would be a P0 incident caught in the staging environment. The fix is environment parity from day one — not a patch after the fact. The reason it happened is that I developed on a Python 3.10+ environment and deployed to Python 3.9, and the pkl format doesn't abstract that boundary. ONNX would have."

28.6 — joblib vs pickle Deserialization Mismatch

The failure: `app.py` used `pickle.load()` to load model artifacts. `train_champion.py` saves with `joblib.dump()`. Error: `_pickle.UnpicklingError: STACK_GLOBAL requires str`. No mention of joblib in the error traceback.

The root cause: joblib serialization extends pickle with extra protocol headers for numpy arrays and memory-mapped objects. Raw `pickle.load()` cannot parse these headers. The fix is trivial (`joblib.load()`), but the error message makes it look like a pickle file corruption.

The lesson: Match your serializer. If it was saved with `joblib.dump()`, load it with `joblib.load()`. The sklearn ecosystem standard is joblib — use it end-to-end. The error signature (`STACK_GLOBAL requires str`) is the recognizable fingerprint for this class of mistake.

28.7 — Interview Summary: What These Failures Actually Show

These failures are not embarrassments — they are evidence:

Failure	Senior engineering behavior demonstrated
DGP default rate overshoot	Verify data distributions before modeling
LightGBM early stopping bug	Audit surprising baselines before accepting them
SHAP pandas index bug	Debug root causes, not symptoms; know your dtypes
Python 3.9 type annotation crash	Environment parity; know the Python version matrix
sklearn pkl version lock-out	Training/serving parity; version-pin artifacts
joblib vs pickle mismatch	Serializer consistency; read the error traceback carefully

The portfolio-level discipline: **none of these were hidden**. Each one is documented in the gate logs, the claim boundary, or the failure archaeology document. A model that looks clean from the outside because failures were buried is more dangerous than one where failures are catalogued.

Section 29 — Gold Pass 5: LSTM Sequence Encoder Experiment

29.1 — What Was Built

Architecture:

- installments_payments.csv (13.6M rows, 339,587 applicants)
- per-row features: days_late, payment_ratio, log_amt_instalment
- last 50 installments per applicant, left-padded with zeros
- 1-layer LSTM (input=3, hidden=64)
- Linear(64→32) + tanh activation → 32-dim embedding
- embeddings appended to 140-feature LightGBM input → 172 features
- LightGBM retrained (same GP2 Optuna hyperparameters, monotone constraints extended with 32 zeros)

Training: PyTorch · Colab T4 GPU · 15 epochs · BCEWithLogitsLoss (pos_weight=11.4 for 8% imbalance) · AdamW(lr=3e-3) · ReduceLROnPlateau · gradient clipping (max_norm=1.0) · batch size 2048

Result:

Metric	Value
GP5 test AUC (calibrated)	0.7264

Metric	Value
GP2 baseline	0.7769
Delta	-0.0505
Verdict	GP2 champion retained

29.2 — Why GP5 Did Not Win

- **35% of applicants have zero installment history** — no installment record in the CSV → zero-padded sequences → the LSTM encodes noise, not signal, for 65,639 of 184,506 training applicants
- **Scalar aggregates already capture delinquency signal** — `inst\_agg.parquet` features (INST_LATE_RATIO, INST_PAY_RATIO) are among the top-7 SHAP features in GP2; the LSTM learned a redundant representation
- **Supervised LSTM on imbalanced labels** — training directly on TARGET (8.1% positive) with only 50-timestep sequences may not converge to embeddings that are more informative than handcrafted aggregates

29.3 — Interview Q&A: Deep Learning Component

Q — "Walk me through a neural component you built in this project."

"In Gold Pass 5, I added an LSTM sequence encoder on top of the existing LightGBM pipeline. The raw data is 13.6 million installment payment rows — one row per payment event per applicant. Rather than relying only on scalar aggregations of these events, I wanted to capture temporal payment behaviour directly.

>

I built a preprocessing script that groups rows by applicant, computes three per-row features — days late, payment ratio, and log instalment amount — takes the last 50 instalments, and left-pads with zeros for shorter histories. This produces an $(N \times 50 \times 3)$ numpy array for 339,000 applicants.

>

The model is a 1-layer LSTM with hidden size 64, followed by a Linear(64→32) projection and tanh activation. I trained it supervised on the default TARGET label using BCEWithLogitsLoss with pos_weight=11.4 to handle the 8% class imbalance. Training ran on Colab T4 GPU in about 10 minutes. The 32-dim output of the projection layer becomes features 141–172 in LightGBM.

>

The result: challenger AUC 0.7264 vs baseline 0.7769 — a -0.0505 drop. The root cause is that 35% of applicants have zero installment history, so their sequences are all zeros; those embeddings inject noise into LightGBM rather than signal. The scalar aggregation features in the existing 140-feature set already captured the key delinquency patterns. The negative result is documented honestly — the GP2 champion is unchanged."

Q — "What would you do differently to make the LSTM work?"

"Three things. First, restrict LSTM features to only the 65% of applicants who have installment history — use the embedding for those applicants and a separate 'no history' indicator feature for the others, rather than zero-padding everyone. Second, try a longer sequence window (100+ payments) and a bidirectional LSTM or self-attention over the sequence — the key default-risk signal may be in early payment patterns, not just the last 50. Third, pre-train the encoder on a reconstruction task (autoencoder on payment sequences) and fine-tune — supervised training on an 8% imbalanced target with a small LSTM may converge to a less informative latent space than unsupervised pre-training."

Q — "Is this the right architecture for tabular + sequence data?"

"For this dataset, probably not. Tree models (LightGBM, XGBoost) consistently outperform neural nets on tabular data when features are engineered well, and the installment aggregates in the feature set are already quite good. A TabNet or a feature tokenizer transformer might extract more from the raw sequences. But the LSTM is architecturally honest — it's a real sequence model, it was trained on real payment histories, and the negative result tells a clear story: handcrafted domain features beat learned embeddings when the domain expert already knows what to aggregate."

29.4 — What GP5 Adds to the Portfolio Story

Even as a negative result, GP5 demonstrates:

- Ability to write PyTorch training loops with proper imbalance handling, early stopping, and gradient clipping
- Understanding of sequence preprocessing (padding, feature engineering at the row level)

- Integration of a neural component with a tree ensemble pipeline
- Honest evaluation: the negative result is documented, not buried
- A concrete answer to "walk me through a neural component you built"

Section 30 — Calibration Forensics: Isotonic Discovery

30.1 — What Happened

The GP2 training pipeline evaluated both Platt (logistic regression) and isotonic regression calibration. The training report stated "Platt selected." The serving code was initially written using the Platt sigmoid formula.

During Cloud Run deployment debugging, the serving probabilities diverged from the training calibrated probabilities by up to 0.53 at some score values — a massive serving gap.

30.2 — Root Cause

Forensic inspection of `champion\_calibrated.pkl`:

```
cal['selected'] # → 'isotonic' (NOT 'platt')
```

The isotonic calibrator was actually selected at training time. The training report was wrong. The `cal\_probs` in the pkl matched `IsotonicRegression.predict()` exactly (max diff = 0.0), and diverged from the Platt sigmoid (max diff = 0.53).

30.3 — The Fix

Rather than re-serialise the sklearn isotonic object (which would reintroduce the version lock), the isotonic calibration was extracted as two numpy arrays:

```
iso_x = iso.X_thresholds_ # shape (134,) – raw probability thresholds
iso_y = iso.y_thresholds_ # shape (134,) – calibrated probability values
np.save('iso_x.npy', iso_x)
np.save('iso_y.npy', iso_y)

# At serve time – exact replication, zero sklearn:
calibrated_prob = float(np.interp(raw_prob, iso_x, iso_y))
```

`np.interp` implements the same piecewise-linear interpolation that `IsotonicRegression.predict()` uses. Max difference between the two: 0.0 across all test values.

30.4 — Interview Answer

*Q — "You said Platt calibration in your writeup, but your code uses isotonic?"

"Yes — this is one of the honest failures I'm glad I found. The training report said Platt was selected, but the pkl had `cal['selected'] = 'isotonic'`. When I deployed the initial serving code using the Platt formula, the calibrated probabilities diverged from training by up to 0.53 — a clear red flag.

>

The forensic fix was simple: read the actual artifact, not the report. Once I confirmed isotonic was the true calibrator, I extracted it as numpy threshold arrays and replicated it with `np.interp`. This eliminated the sklearn version dependency entirely — the calibration now runs as a two-array numpy operation with zero imports from sklearn. ECE on test is 0.0034."

PulseGuard — Interview Defense Document | V4 GP5 + Calibration Forensics | 2026-06-21

Section 27D deployment caveat appended: 2026-06-20

Section 28 "Failures I'm Proud Of" appended: 2026-06-20

Section 29 GP5 LSTM experiment appended: 2026-06-21

Section 30 Calibration forensics appended: 2026-06-21

Sections 1–21: project-specific defense. Sections 22–26: adversarial follow-ups, ML concepts, implementation probes, behavioral. Section 27: fraud / MLOps / risk scoring role expansion + deployment caveat. Section 28: failure archaeology. Section 29: GP5 deep learning component. Section 30: isotonic calibration discovery.